

11-1 Introduction to Consumer Analysis and CRM in Marketing

Chapter Introduction

In this chapter, we will explore consumer analysis and CRM (Customer Relationship Management) within marketing strategy. We will introduce foundational concepts related to these areas and demonstrate several representative methods for implementing and conducting analysis using Python. These methods form the basis of marketing and consumer analysis. By mastering the skills presented here, you will be able to apply a hands-on approach to gain a foundational understanding of consumer behavior analysis.

Goal

- Understand the basic approaches for analyzing CRM data in marketing and to implement representative methods (such as decile analysis, RFM analysis, and churn analysis) using Python for analysis.
 - Understand how to identify three key points in consumer data analysis.
-

11-1.1 Background and Overview of this Chapter

Keywords: CRM, POS data, defining the purpose of analysis, hypothesis-driven thinking, causality, comparison, hands-on practice

11-1.1.1 Consumer Behavior Analysis and CRM

In this chapter, we will learn methods for understanding consumer behavior and about **CRM**. CRM stands for **Customer Relationship Management**. As the name suggests, it is a marketing approach where companies providing services build trust with customers, aiming to improve customer satisfaction and loyalty, and ultimately increase revenue.

Why has CRM become increasingly important in recent years? The background for the growing emphasis on CRM is that traditional mass marketing approaches, which target the general public, have become less effective. There is now a need to offer tailored suggestions and services based on each customer's preferences and needs. Mass

marketing strategies, such as TV advertisements, used to be effective when broadcasted widely. However, nowadays, the focus has shifted from mass advertising to defining more specific user profiles and targeting a narrower audience for a more focused approach.

In recent years, technological advancements (such as cloud computing, improvements in machine processing speeds, and reduced costs for data storage) have created an environment where customer data can be gathered along various dimensions.

Representative types of customer data include demographic information such as gender, age, occupation, address, family structure, workplace, and income. Additionally, there is data on online behavior (access log data) and **POS (Point of Sales) data** from purchases made at convenience stores or supermarkets. Furthermore, data such as login information for services, advertisements viewed or clicked by the individual, and location data are also being accumulated.

By accumulating and integrating various types of data related to customer behavior, it becomes possible to understand what the customer is seeking, their preferences, and their life stage (such as being a student, job hunting, getting married, having children, or retiring). This enables the provision of services tailored to each customer. Given this background, CRM and data related to CRM are becoming increasingly important.

In this lecture, we will introduce various methods for analyzing users based on POS data accumulated when consumers make purchases on e-commerce sites, and explain how to utilize the results effectively.

Side Note 1: Different Examples of Data Utilization Related to CRM

CRM data is primarily used to improve services for customers, but recently, in the financial sector, it has started to be used to determine whether transactions are related to criminal activities. This is part of the field known as AML (Anti-Money Laundering). Conventionally, detecting fraudulent transactions was often done manually, and automation was slow to develop. However, by integrating CRM customer information with AI (machine learning), systems are being developed that can more precisely detect such activities, helping to reduce the costs associated with audits. This represents a different use of CRM data compared to conventional applications.

✓ 11-1.1.2 Three Key Points in Consumer Behavior Data Analysis

In this chapter, we will explain representative methods for analyzing consumer behavior, along with Python implementations, so that you can apply them yourself. However, it is

important to note that each method is just a framework for analysis. So, what is the most important thing when conducting analysis? The answer is, as expected, having a clear analysis objective and executing it in line with the marketing strategy of the company or business. While marketing strategy is a broader topic that will be addressed later, here we will first focus on the importance of having a clear analysis objective.

While it would be ideal to have access to various types of consumer data as mentioned above, there are often cases where the desired data is not available. Additionally, it is nearly impossible to collect all data about customers. Trying to achieve this would require setting up IT infrastructure (which will be briefly discussed in a later section), and it could involve significant time and costs.

Furthermore, as we will explain later using specific POS data, even if the sales of a product are increasing, it is unlikely that the cause can be identified from the POS data alone. With limited data, the insights that can be gained are also limited. While it may seem obvious, conducting data analysis without a clear objective or blindly manipulating data without proper design makes it difficult to derive meaningful insights.

So, what should be done? Here, three key points will be discussed.

The first point is to "have a hypothesis."

The second point is whether the analysis leads to "actions that can trigger change through experimentation".

The third and final point is "comparison."

1. Hypothesis-driven objectives
2. Actionable insights
3. Method comparisons

A very simple example would be considering a product recommendation for a customer segment group (let's call it X). Suppose you are unsure whether it is better to recommend product A, which has no purchase history, or product B. In this case, you might discover that other customers with similar preferences and behavior history to those in segment X have previously purchased product A. This leads to the hypothesis that recommending product A to customers in the X group might increase the likelihood of them making a purchase. This is the first key point. (However, this is just a simple example; in practice, much more complex approaches are used.)

Next, regarding the second point, actionable insights, the actual outcome of the hypothesis cannot be known until it is tried. Therefore, being able to conduct an "experiment" to test the hypothesis is crucial. This means recommending product A to

the customer group X (via a website, direct contact, email, etc.). This is the concrete action. Sometimes, analysis is conducted, but it may be unclear how to apply the findings or what improvements will result. This situation occurs when there is no connection to action or causality. In the current example, the analysis of the customer group's behavior and the experiment of influencing that behavior through the recommendation links to change, which is why this is the second key point, "actionable insights."

Finally, the third important point in analysis is method comparisons. Analysis is essentially synonymous with comparison and is critical. There are various methods and dimensions for comparison, but in the example above, you could randomly select a few customers from segment X to form a group that will not receive any recommendation (this is called the control group). The group that receives the recommendation for product A is referred to as the test group. By analyzing the differences in purchase rates between the test group and the control group, you can determine whether the recommendation was effective. This is similar to an A/B test on the web, where you define comparison groups and adopt the strategies that show positive results.

Other methods of comparison may involve not just between groups, but also comparing sales to the previous year or previous month. By making such comparisons, you can analyze which strategies worked and which did not, allowing for data-driven improvements. However, it is important to note that not everything should be subjected to A/B testing. For example, if the user base is too small or if it is clear that product A is superior, running an A/B test may not yield meaningful results. Additionally, if the comparison group is not selected correctly, there is a risk of falsely attributing an effect, so caution is required.

By keeping these three points in mind during analysis, it becomes possible to run the **PDCA cycle (Plan-Do-Check-Act)**, which is often mentioned as a framework for continuous improvement: hypothesis -> experiment -> verification -> improvement. If analysis cannot be done within this cycle, it can be said that it is of limited value.

However, in this lecture, since we cannot conduct experiments or verifications, we will focus on implementing Python to analyze data with a hypothesis in mind and make comparisons, all while keeping these points in mind.

Reference: "Standard of Digital Marketing" (Digital Marketing no Joseki)

11-1.1.3 Identify Issues

As mentioned earlier, it is important to clarify the analysis objectives in line with the marketing strategy, and three key points for analysis have been introduced. However, there may be cases where it is unclear what the objectives should be or what the specific challenges are. One approach to address this is the use of sales factor decomposition, often introduced in books on logical thinking, to break down the causes of sales performance.

Specifically, the decrease in sales can be attributed to various factors, such as a reduction in new customer acquisition (inflow), poor retention rates (heavy outflow), or a decrease in sales per customer. These factors can be broken down and analyzed. This will be implemented using Python, with the analysis visualized through hands-on exercises. The key point here is "comparison". We will look at several methods to easily aggregate and visualize sequential data.

✓ 11-1.1.4 There is no silver bullet in data analysis

The three key points of analysis and how to identify issues have been discussed so far. These are often mentioned in various sources, so many may already be familiar with them. However, while it is easy to talk about these concepts, putting them into practice is much more challenging.

Once the hypotheses or issues are identified, the next step is simply implementing them in Python. As long as coding skills improve, the implementation can be executed quickly. (However, a key objective of this course is for participants to reach a level where they can easily think of implementations and take actions immediately.)

When implementing methods to verify the hypotheses, keep in mind that "there is no silver bullet". This is a common saying in system development, which concept applies equally to data analysis for marketing. It means that there is no one-size-fits-all solution or a universal method to address every challenge or problem. There are no flashy tricks; instead, it's crucial to carefully examine, solve, and improve each individual issue.

This course will introduce representative data analysis approaches, but they are merely frameworks. There is no "silver bullet" that guarantees perfection. It is essential to examine the data carefully, and sometimes, it may be necessary to look at the raw data directly.

In fact, Workman Co., Ltd., a company that sells workwear and other apparel, has required all its employees to use Excel for analysis, which has resulted in strong sales performance. Excel is an excellent tool for viewing data in a straightforward manner, and

the habit of thoroughly reviewing data likely contributes to their sales increase. Similarly, engineers who built their own financial trading systems have also used Excel for quick data aggregation and visualization when dealing with small datasets.

Data analysis, being a meticulous task, is sometimes likened to "civil engineering". Many skilled engineers and data scientists I have met have emphasized the importance of looking at raw data. While this work may not be glamorous, it has proven to produce results, lending credibility to the idea that there is no silver bullet.

In this course, while we will focus on using Python to handle large-scale data processing and systematization, the need to examine raw data and the importance of carefully analyzing data remain consistent. Therefore, such cases will also be covered.

11-1.1.5 The Overall Structure of This Chapter

The overall structure of this chapter is as follows. First, we will introduce the importance of "strategy" and objectives for improving customer service, rather than focusing solely on data analysis. We will briefly cover marketing strategies, KPIs, and data strategies (such as IT infrastructure). While this is not about implementation, it is conceptually important, and we will also explain the significance of setting up an environment for data collection. Next, we will proceed with data preparation, acquisition, validation, loading, and preprocessing so that you can get hands-on experience. Then, we will introduce basic exploratory data analysis on the loaded data and learn representative methods using CRM data (such as RFM analysis and churn analysis) with Python implementations.

There will be practice questions along the way, and at the end, a comprehensive question. Please make sure to implement and try out the exercises. Also, if you come up with any hypotheses you would like to test, feel free to implement and verify them. That is what will truly strengthen your analytical skills.

✓ 11-1.2 Marketing Strategy and Data Analysis

Keywords: Marketing Strategy, KPI, KGI, Data Strategy, IT Infrastructure

✓ 11-1.2.1 Marketing Strategy in Businesses

In the previous section, the important points about utilizing CRM data and other sources to understand consumer behavior were discussed. However, an even more fundamental concept is to first have a clear understanding of the business vision and philosophy of

the company or business division. It is also crucial to determine the business direction, identify key areas to strengthen, and make decisions about which businesses to focus on or potentially withdraw from. These decisions are part of a company-wide strategy or business strategy, typically discussed at the executive level. While this might seem somewhat distant, here we will briefly cover the marketing strategy, which is a component of that broader strategy.

"GLOBIS MBA Marketing" defines marketing as "activities that create a system to encourage purchases, while focusing on customer needs and customer satisfaction". While some companies may have a dedicated marketing department, marketing is typically a department that interacts with almost all other departments in the company. For example, research and development or manufacturing departments involved in developing a product must also consider the consumer's perspective, such as what the users want and what potential needs exist.

Of course, it is crucial to pursue cutting-edge technology. However, no matter how new and excellent the technology is, if it does not ultimately provide what buyers or consumers need, it becomes irrelevant. In Japan, the focus has often been more on manufacturing than on marketing thinking, which sometimes results in a lack of consideration for what users truly want, even though the technical capabilities are not inferior to those overseas. In this sense, marketing thinking to understand customer needs is extremely important. Moreover, departments related to logistics, procurement, and even human resources must adopt this consumer-centric mindset, as it is an essential skill for people in any department.

In the following sections, we will learn about consumer analysis and CRM using data. These are parts of the marketing strategy and serve as the fundamental frameworks for understanding customer behavior. Specifically, we will cover techniques like decile analysis, RFM analysis, and churn analysis. The goal of this course is for you to be able to implement and analyze these techniques through hands-on practice.

Reference: "GLOBIS MBA Marketing,"

✓ 11-1.2.2 What is KPI?

When considering marketing strategies, **KPIs** and **KGIs** become important. Many people have likely heard of these terms. KPI stands for **Key Performance Indicators** and these are the numbers that should be tracked based on the company's strategy. Simple examples include "sales revenue" or "number of new customer acquisitions." The reason these are important is that without such numbers, it becomes difficult to assess whether

the strategy is working or not, and decisions may be made based on intuition rather than data.

Additionally, KGI, which can broadly be considered as part of KPI, stands for **Key Goal Indicators**. It refers to the numerical goal to be achieved. For example, sales revenue is a KPI, but setting a target figure, such as an annual sales goal of 5B USD, would be an example of a KGI.

By determining the numbers to track, verifying whether they can be achieved, and making improvements, it becomes possible to visualize more objective metrics and run the PDCA cycle effectively.

Incidentally, it is said that sales representatives at Keyence Corporation, a well-known company in sales, have hundreds of KPIs. As mentioned in the reference books, this is an extreme example. However, the mindset of setting goals, tracking numbers, constantly considering why a goal wasn't achieved, and thinking about how to improve is very important.

On the other hand, KPIs are merely a means to an end and should be used as a tool to achieve objectives. It is important to avoid setting rigid KPIs that cannot be changed and placing excessive emphasis on them. The key is understanding why KPIs are being set and managed. If not done correctly, it can lead to heading in the wrong direction. For example, achieving sales targets doesn't mean pushing sales too aggressively. Such actions can affect future sales and reputation, so it's important to set and track KPIs carefully.

Later, you will learn how to implement Python to analyze metrics such as sales trends from POS data and churn rates (cancellation rates). Of course, depending on the data volume, these metrics can sometimes be calculated in Excel, so feel free to use the appropriate tool for the task at hand.

✓ 11-1.2.3 Data Strategy and IT Infrastructure

So far, we have discussed marketing strategies and KPIs for managing these metrics. However, without data to track these KPIs, nothing can be done. Therefore, it is essential to properly collect data related to KPIs, and it is also important to develop a "data strategy" and an IT infrastructure roadmap for this purpose (as mentioned in Chapter 10 of *Data-Driven Marketing*, which is also a helpful reference).

At the beginning, there is no need for a large-scale system, and it might be possible to start with Excel. As mentioned earlier, companies like Workman use such an approach.

However, when there is a need to process a large amount of data, or when manual tracking becomes unmanageable, it becomes necessary to develop IT infrastructure and systems, and to create a roadmap for this.

There are cases where, despite having established business procedures and workflows, companies face large amounts of manual Excel work (copy-pasting, modifying functions, adding sheets, etc.). The author has witnessed such situations in various companies. Many businesses are overwhelmed by manual tasks like Excel work and struggle to allocate time for original thinking and work. When the scale of data processing becomes significant, it is necessary to consider software development or cloud-based systems rather than relying on manual Excel systems.

The purpose of a data strategy is to improve customer service, and as mentioned earlier, a variety of data can be collected today. However, there have been many cases where gathering data blindly does not serve its purpose, so it is essential to accumulate data that aligns with the marketing strategy and objectives.

Side Note 2: The Importance of Clarifying Business Processes (Process Mining)

Recently, digital transformation (DX), data analysis, and AI have been gaining attention. However, rather than jumping directly into these practices, the first important step is to clearly understand business processes and workflows. It is crucial to consider where digitalization can or cannot be applied, where systems can be implemented, whether there are unnecessary processes or tasks, and where the quality of customer service can be improved.

Without understanding the business, it is not possible to advance systematization or DX. Efficiency gains are based on existing business, so considering DX or AI comes secondary. This may sound like an obvious point, but the reason many projects are facing issues is likely because the means have become the end goal.

The first step is to clarify the business flow, improve inefficient tasks that are manually done or duplicated by systematizing them, and then accumulate the related data. Afterward, the next phase involves using machine learning and AI to learn from this large amount of data.

In terms of improving business processes, there is a field called process mining, which is gaining attention.

Reference: "The Shock of Process Mining: Why Siemens, BMW, and Uber Are Seriously Engaged in It"

✓ 11-1.3 Data Loading and Processing Handled in This Chapter

Keywords: POS Data, Basket and Register Data, Web Scraping

✓ 11-1.3.1 Target POS Data and Preparation

Now that we've covered the conceptual aspects, let's move on to practical implementation where you will actually work with Python to analyze customer behavior. In this chapter, we will focus on analyzing purchase data, which is a part of CRM (Customer Relationship Management) commonly used in marketing analysis. This type of data is also referred to as transaction data.

To visualize purchase data, think of a scenario where you are shopping at a supermarket or convenience store. When you pay at the register, a receipt is printed, and this receipt data is what we are working with. If the data also includes customer IDs and the purchase history is available for each customer, it is known as ID-tagged or **POS (Point of Sale) data**.

The data we will work with here comes from corporate purchases rather than individual consumers, but the analysis framework is essentially the same. You can load the data from the URL provided below using Pandas (this data is large, with more than 500,000 entries, so it may take some time to load).

While analyzing 500,000 rows of data can be done in Excel, and it may be quicker for summarizing or graphing, this course will focus on preparing you to handle datasets of over a million rows. We aim to give you an approach that will be useful for automating the analysis and scaling to larger systems, which is why we will use Python for this analysis.

<https://archive.ics.uci.edu/ml/datasets/online+retail>

In this chapter, we will use various libraries necessary for analysis. We will proceed with the assumption that the following libraries have been imported:

```
# Please load the following libraries in advance as they will be used
import numpy as np
import pandas as pd

import datetime

# Visualization library
import matplotlib.pyplot as plt
```

```
import seaborn as sns
sns.set()
%matplotlib inline

# Display up to the third decimal place
%precision 3

'%.3f'
```

The website with the purchasing data is specified, and we use ExcelFile to read the Excel file with Pandas and handle it as a DataFrame. This part is a review of Pandas, so if you are unsure, it is a good idea to review it.

```
# Download the data from the UCI website
!wget "http://archive.ics.uci.edu/static/public/352/online+retail.zip"
# Extract the downloaded data.
!unzip "./online+retail.zip"
# Load the extracted Excel data.
file_url = './Online Retail.xlsx'
all_online_retail_data = pd.ExcelFile(file_url)
```

```
--2025-03-16 19:08:24-- http://archive.ics.uci.edu/static/public/352/online+retail.zip
Resolving archive.ics.uci.edu (archive.ics.uci.edu)... 128.195.10.252
Connecting to archive.ics.uci.edu (archive.ics.uci.edu)|128.195.10.252|:80... cc
HTTP request sent, awaiting response... 200 OK
Length: unspecified
Saving to: 'online+retail.zip'

online+retail.zip      [      <=>          ] 22.62M  30.4MB/s   in 0.7s

2025-03-16 19:08:25 (30.4 MB/s) - 'online+retail.zip' saved [23715478]

Archive:  ./online+retail.zip
  extracting: Online Retail.xlsx
```

If the Excel file contains multiple sheets, the `parse` method can be used to specify the sheet name to load.

```
# Specify a sheet in an Excel file
online_retail_data_table = all_online_retail_data.parse('Online Retail')
```

Then, let's quickly review the data that has been loaded.

```
online_retail_data_table.head()
```

	InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice	Customer
0	536365	85123A	WHITE HANGING HEART T- LIGHT HOLDER	6	2010-12-01 08:26:00	2.55	17850
1	536365	71053	WHITE METAL LANTERN	6	2010-12-01 08:26:00	3.39	17850

```
online_retail_data_table.info()
```

```
<class 'pandas.core.frame.DataFrame'>  
RangeIndex: 541909 entries, 0 to 541908
```

```
Data columns (total 8 columns):
```

```
#   Column              Non-Null Count  Dtype  ---  ---  ---  
0   InvoiceNo          541909 non-null    object  
1   StockCode          541909 non-null    object  
2   Description         540455 non-null    object  
3   Quantity           541909 non-null    int64  
4   InvoiceDate         541909 non-null    datetime64[ns]  
5   UnitPrice          541909 non-null    float64  
6   CustomerID         406829 non-null    float64  
7   Country            541909 non-null    object
```

```
dtypes: datetime64[ns](1), float64(2), int64(1), object(4)
```

```
memory usage: 33.1+ MB
```

The columns are as follows. By the way, **basket** refers to a single purchase, similar to the image of a shopping basket. One receipt corresponds to one basket.

- **InvoiceNo**: The purchase (basket) number
- **StockCode**: The product code
- **Description**: Name of the product
- **Quantity**: The quantity of the product
- **InvoiceDate**: The date and time of the purchase
- **UnitPrice**
- **CustomerID**
- **Country**: Country name

Now, looking at this data alone, it may be difficult to imagine the products. Probably because even when looking at the product names, it may be hard to form an image. So

let's input some product names as search terms and search for them.

For example, the first item, "WHITE HANGING HEART T-LIGHT HOLDER," if you search it on Google, you will likely see an image of an object shaped like a heart. Similarly, if you search for "KNITTED UNION FLAG HOT WATER BOTTLE," you will find an image of a cover for a hot water bottle.

It seems that they sell household goods and other similar items, and according to the UCI site providing this data, it is explained as follows:

"This is a transnational data set which contains all the transactions occurring between 01/12/2010 and 09/12/2011 for a UK-based and registered non-store online retail. The company mainly sells unique all-occasion gifts. Many customers of the company are wholesalers."

It appears that this is an online retail purchase data from an e-commerce site based in the UK.

<https://archive.ics.uci.edu/ml/datasets/online+retail>

Reference: How to automate the collection of image data from the web (Web scraping)

As mentioned earlier, there are image data available on the web, but manually inputting, searching, verifying the images, and downloading them is very time-consuming. Therefore, we will use a technique called web scraping to retrieve image data from the web. First, create a directory for storing the data (pictures) and move to that directory.

```
mkdir pictures
```

```
cd pictures/
```

```
/content/pictures
```

Let's create a list of products. This part of the implementation is also a review of Python and Pandas.

```
product_list_s = online_retail_data_table.Description.value_counts().sort_value  
product_list_s
```

	count
Description	
WHITE HANGING HEART T-LIGHT HOLDER	2369
REGENCY CAKESTAND 3 TIER	2200
JUMBO BAG RED RETROSPOT	2159
PARTY BUNTING	1727
LUNCH BAG RED RETROSPOT	1638
...	...
dotcom sales	1
LIGHT DECORATION BATTERY OPERATED	1
ASSORTED TUTTI FRUTTI ROUND BOX	1
RUBY GLASS CLUSTER NECKLACE	1
PINK BERTIE MOBILE PHONE CHARM	1

4223 rows × 1 columns

dtype: int64

```
product_list = list(product_list_s.index)
```

Let's create a list of products. This part of the implementation is also a review of Python and Pandas.

```
!pip install icrawler
```

```
Collecting icrawler
  Downloading icrawler-0.6.9-py3-none-any.whl.metadata (2.8 kB)
Requirement already satisfied: beautifulsoup4 in /usr/local/lib/python3.10/dist-
Collecting bs4 (from icrawler)
  Downloading bs4-0.0.2-py2.py3-none-any.whl.metadata (411 bytes)
Requirement already satisfied: lxml in /usr/local/lib/python3.10/dist-packages (
Requirement already satisfied: pillow in /usr/local/lib/python3.10/dist-packages
Requirement already satisfied: pyyaml in /usr/local/lib/python3.10/dist-packages
Requirement already satisfied: requests in /usr/local/lib/python3.10/dist-packag
Requirement already satisfied: six in /usr/local/lib/python3.10/dist-packages (f
Requirement already satisfied: soupsieve>1.2 in /usr/local/lib/python3.10/dist-p
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.10/dist-pa
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.10/d
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.10/d
Downloading icrawler-0.6.9-py3-none-any.whl (34 kB)
Downloading bs4-0.0.2-py2.py3-none-any.whl (1.2 kB)
```

```
Installing collected packages: bs4, icrawler  
Successfully installed bs4-0.0.2 icrawler-0.6.9
```

Since retrieving all products would take a long time, let's limit it to 10 products and download three images for each search term. The reason for downloading three images is that one image alone may not provide enough context.

```
product_list[:10]  
  
['WHITE HANGING HEART T-LIGHT HOLDER',  
 'REGENCY CAKESTAND 3 TIER',  
 'JUMBO BAG RED RETROSPOT',  
 'PARTY BUNTING',  
 'LUNCH BAG RED RETROSPOT',  
 'ASSORTED COLOUR BIRD ORNAMENT',  
 'SET OF 3 CAKE TINS PANTRY DESIGN ',  
 'PACK OF 72 RETROSPOT CAKE CASES',  
 'LUNCH BAG  BLACK SKULL.',  
 'NATURAL SLATE HEART CHALKBOARD ']
```

The following loops through each search term, performs a web image search based on it, and executes web scraping to download the images to this environment.

```
for key_word in product_list[:10]:  
    from icrawler.builtin import BingImageCrawler  
    print('key word:{0}'.format(key_word))  
    crawler = BingImageCrawler(storage={"root_dir": key_word})  
    crawler.crawl(keyword=key_word, max_num=3)
```

```
key word:WHITE HANGING HEART T-LIGHT HOLDER  
key word:REGENCY CAKESTAND 3 TIER  
key word:JUMBO BAG RED RETROSPOT  
key word:PARTY BUNTING  
key word:LUNCH BAG RED RETROSPOT  
key word:ASSORTED COLOUR BIRD ORNAMENT  
key word:SET OF 3 CAKE TINS PANTRY DESIGN  
key word:PACK OF 72 RETROSPOT CAKE CASES  
key word:LUNCH BAG  BLACK SKULL.  
key word:NATURAL SLATE HEART CHALKBOARD
```

Now that the download is complete, let's take a look at the actual image data.

After downloading 3 images for each of the 10 products, we will display them in a 3x10 grid format in the Jupyter Notebook. By using the `glob` library, we can retrieve all file paths under the specified directory.

```
import os
import glob
os.chdir("/content/pictures/")
dir_list = glob.glob(".*")
dir_list

['./PACK OF 72 RETROSPOT CAKE CASES',
 './JUMBO BAG RED RETROSPOT',
 './WHITE HANGING HEART T-LIGHT HOLDER',
 './PARTY BUNTING',
 './SET OF 3 CAKE TINS PANTRY DESIGN ',
 './LUNCH BAG BLACK SKULL.',
 './ASSORTED COLOUR BIRD ORNAMENT',
 './LUNCH BAG RED RETROSPOT',
 './NATURAL SLATE HEART CHALKBOARD ',
 './REGENCY CAKESTAND 3 TIER']
```

The following is the implementation for displaying the images from the directory where they were retrieved. It uses `add_subplot` to display multiple images and `Image` for rendering the images.

```
from PIL import Image

plot_num = 1
fig = plt.figure(figsize=(10,13),dpi=100)
for dir in dir_list:
    print(dir)
    cnt = 1
    for pic_path in glob.glob(dir+'/*'):
        ax=fig.add_subplot(10, 3, plot_num)
        ax.axis("off")
        im = Image.open(pic_path)
        plt.title(dir+'_'+str(cnt), fontsize=5)
        plt.imshow(im)
        plot_num = plot_num + 1
        cnt = cnt + 1
```

- ./PACK OF 72 RETROSPOT CAKE CASES
- ./JUMBO BAG RED RETROSPOT
- ./WHITE HANGING HEART T-LIGHT HOLDER
- ./PARTY BUNTING
- ./SET OF 3 CAKE TINS PANTRY DESIGN
- ./LUNCH BAG BLACK SKULL.
- ./ASSORTED COLOUR BIRD ORNAMENT
- ./LUNCH BAG RED RETROSPOT
- ./NATURAL SLATE HEART CHALKBOARD
- ./REGENCY CAKESTAND 3 TIER

./PACK OF 72 RETROSPOT CAKE CASES_1



./JUMBO BAG RED RETROSPOT_1



./WHITE HANGING HEART T-LIGHT HOLDER_1



./PARTY BUNTING_1



./SET OF 3 CAKE TINS PANTRY DESIGN_1



./LUNCH BAG BLACK SKULL_1



./ASSORTED COLOUR BIRD ORNAMENT_1



./LUNCH BAG RED RETROSPOT_1



./PACK OF 72 RETROSPOT CAKE CASES_2



./JUMBO BAG RED RETROSPOT_2



./WHITE HANGING HEART T-LIGHT HOLDER_2



./PARTY BUNTING_2



./SET OF 3 CAKE TINS PANTRY DESIGN_2



./LUNCH BAG BLACK SKULL_2



./ASSORTED COLOUR BIRD ORNAMENT_2



./LUNCH BAG RED RETROSPOT_2



./PACK OF 72 RETROSPOT CAKE CASES_3



./JUMBO BAG RED RETROSPOT_3



./WHITE HANGING HEART T-LIGHT HOLDER_3



./PARTY BUNTING_3



./SET OF 3 CAKE TINS PANTRY DESIGN_3



./LUNCH BAG BLACK SKULL_3



./ASSORTED COLOUR BIRD ORNAMENT_3



./LUNCH BAG RED RETROSPOT_3



This should give a general idea of the types of products being handled.

In conclusion, using the web scraping technique, you were able to specify your desired search terms in a search engine, search for images, and confirm that the images were properly downloaded. However, it is important to note that there may be legal issues involved with accessing certain websites, so caution should be exercised when implementing or using this technique.

✓ 11-3.3.2 The Purpose of Data Analysis and the Scope of Data Analysis

Now, let's return to the topic of data. The most important question is, what do you want to achieve with this purchasing data? In this lecture, the goal is to help you acquire the basic framework for consumer behavior analysis by introducing various approaches and implementing them in Python. However, these approaches are simply templates for consumer data analysis, and whether they can be immediately applied depends on the situation.

So, how can we utilize data? As mentioned earlier, it is important to conduct analysis with a clear analytical purpose or hypothesis in mind. It is ok to approach the data with a vague thought of "Can I find something interesting in this data?", if there is unlimited time or if the analysis is exploratory and time-limited. However, there are many cases where you end up manipulating the data without understanding what you were trying to achieve in the first place. Therefore, always remember the question, "What is the purpose of analyzing this data?"

In this lecture, we will specifically introduce analysis methods with the goal of "identifying" high-value customers, converting regular customers into high-value ones, or identifying customers at risk of churn and appropriately addressing them to prevent churn.

For instance, there may be cases where a company wants to create a membership card to increase customer loyalty but is unsure about how to set the right amount for membership benefits. In such cases, analysis can be done to segment customers and determine optimal pricing. Additionally, for product recommendations or identifying customers who frequently purchase discounted items, actions such as offering coupons for products they haven't yet purchased might be effective. On the other hand, there might be customers for whom discounts have no effect, and the data from this can be accumulated and used for future decisions. Furthermore, if this data pertains to a

physical store, adjustments such as changing shelf placements or determining the appropriate number of staff to hire could also be important considerations.

From another perspective, consider the purpose of data analysis in terms of inventory management. Let's say, in the upcoming months, you need to decide how much to order for each product or category, how much stock to keep, or whether to reduce your orders. In such a case, decisions may be made based on whether a product is not selling well and should be reduced, or whether a product's sales are rising and should be increased. This is related to demand forecasting. For example, if the sales of Product A are declining, instead of deciding to halt production, the data might reveal that the decline is specific to the UK, while other countries still have demand and the sales are gradually increasing. Through the methods covered in this lecture, if the data is available, it will be possible to analyze the sales of products by company, country, or region quickly.

Furthermore, by carefully considering the data strategy mentioned above and building an integrated data platform, continuously conducting experiments and hypothesis testing, better and quicker decision-making will be possible, allowing you to gain a competitive edge over other companies.

As such, various purposes and ways to utilize data can be considered. However, while data analysis can help identify customers or track product sales trends, determining the appropriate actions or conducting experiments is a separate matter. While hints can be derived from purchase data, it is crucial to translate these insights into actual actions. Without taking action, the analysis holds no meaning. Past data alone cannot solve everything. As mentioned earlier in Section 11-1.1.2, these points should be kept in mind when conducting analyses.

✓ 11-1.3.3 Data Preprocessing

Once the objectives and hypotheses are defined, the next step is to check and preprocess the data. It is often said that 80-90% of data analysis time is spent on preprocessing (including tasks like machine learning or deep learning implementation). While these tasks might seem tedious, they are essential for conducting proper data analysis.

In this case, the analysis is customer-centric, so we will focus on the records where the CustomerID field has data. As for the data itself, there is a note stating that any

InvoiceNo starting with "C" refers to a cancellation. Therefore, we will first remove those records. Additionally, we will remove any InvoiceNo entries where the first character is not a number. We will write the code for these preprocessing steps.

From here, the basic data operations on Pandas DataFrames, such as filtering data, and understanding how to use the map function and lambda functions, are considered prerequisites.

The following process extracts the "C" for cancellations from the `InvoiceNo`, takes the first character, and adds it as a flag in a new column.

```
online_retail_data_table['cancel_flg'] = online_retail_data_table.InvoiceNo.map
```

Looking at the following result, there are also characters like 5 and A besides C. 5 is for regular purchases, and A, when checking the records, is labeled as "Adjust bad debt," so it seems appropriate to remove this as well.

```
online_retail_data_table.cancel_flg.value_counts()
```

	count
cancel_flg	
5	532618
C	9288
A	3

dtype: int64

```
online_retail_data_table[online_retail_data_table.cancel_flg=="A"]
```

	InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice	Cus
299982	A563185	B	Adjust bad debt	1	2011-08-12 14:50:00	11062.06	
299983	A563186	B	Adjust bad debt	1	2011-08-12 14:51:00	-11062.06	
299984	A563187	B	Adjust bad debt	1	2011-08-12 14:52:00	-11062.06	

Additionally, from the data information, we can see that some entries are missing the `CustomerID`. (There are approximately 540,000 total records with non-null values, but only around 400,000 non-null entries for `CustomerID`.)

```
online_retail_data_table.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 541909 entries, 0 to 541908
Data columns (total 9 columns):
#   Column          Non-Null Count  Dtype
---  -
0   InvoiceNo        541909 non-null object
1   StockCode       541909 non-null object
2   Description     540455 non-null object
3   Quantity        541909 non-null int64
4   InvoiceDate      541909 non-null datetime64[ns]
5   UnitPrice       541909 non-null float64
6   CustomerID      406829 non-null float64
7   Country         541909 non-null object
8   cancel_flg      541909 non-null object
dtypes: datetime64[ns](1), float64(2), int64(1), object(5)
memory usage: 37.2+ MB
```

✓ Side Note 3: Copying the Data

Now, we will perform data filtering. However, since we will use the data in its unfiltered state later, we will make a copy of the data to keep it. When copying, we implement it as follows, so please take note. The reason for using this will be explained in section 11-1.4 later.

We are performing preprocessing to extract records where `CustomerID` is not null. Additionally, since `CustomerID` is in decimal format, which feels a bit odd, we will convert the data type to int.

```
tmp_online_retail_data_table = online_retail_data_table.copy()
tmp_online_retail_data_table = tmp_online_retail_data_table[tmp_online_retail_data_table.CustomerID != tmp_online_retail_data_table.CustomerID.isnull().values]
tmp_online_retail_data_table.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 406829 entries, 0 to 541908
Data columns (total 9 columns):
#   Column          Non-Null Count  Dtype
---  -
0   InvoiceNo        406829 non-null object
1   StockCode       406829 non-null object
2   Description     406829 non-null object
3   Quantity        406829 non-null int64
4   InvoiceDate      406829 non-null datetime64[ns]
5   UnitPrice       406829 non-null float64
6   CustomerID      406829 non-null int64
7   Country         406829 non-null object
8   cancel_flg      406829 non-null object
dtypes: datetime64[ns](1), float64(2), int64(2), object(4)
memory usage: 28.2+ MB
```

```

0  InvoiceNo      406829 non-null object
1  StockCode     406829 non-null object
2  Description   406829 non-null object
3  Quantity      406829 non-null int64
4  InvoiceDate    406829 non-null datetime64[ns]
5  UnitPrice     406829 non-null float64
6  CustomerID    406829 non-null int64
7  Country       406829 non-null object
8  cancel_flg    406829 non-null object
dtypes: datetime64[ns](1), float64(1), int64(2), object(5)
memory usage: 31.0+ MB

```

For now, let's set aside the data above and work with the data processed as shown below.

The following is preprocessing to extract records that are not canceled (those starting with the digit 5) and records where `CustomerID` is not null.

```

online_retail_data_table = online_retail_data_table[(
    online_retail_data_table.cancel_flg == '5') & (online_retail_data_table.Cus

```

As with the previous step, the `CustomerID` is being changed to an int type.

```

online_retail_data_table.CustomerID = online_retail_data_table.CustomerID.astype

```

```

<ipython-input-21-4fc111a88fdc>:1: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

```

See the caveats in the documentation: <https://pandas.pydata.org/pandas-docs/stab>
`online_retail_data_table.CustomerID = online_retail_data_table.CustomerID.asty`

```

online_retail_data_table.info()

```

```

<class 'pandas.core.frame.DataFrame'>
Index: 397924 entries, 0 to 541908
Data columns (total 9 columns):
#   Column          Non-Null Count  Dtype
---  -
0   InvoiceNo        397924 non-null object
1   StockCode        397924 non-null object
2   Description      397924 non-null object
3   Quantity         397924 non-null int64
4   InvoiceDate       397924 non-null datetime64[ns]
5   UnitPrice        397924 non-null float64
6   CustomerID       397924 non-null int64
7   Country          397924 non-null object

```

```
8    cancel_flg    397924 non-null object  
dtypes: datetime64[ns](1), float64(1), int64(2), object(5)  
memory usage: 30.4+ MB
```

The simple data preprocessing is now complete. It might be a bit tedious, and there may be even more challenging cases for data preprocessing. However, since it will affect the results of future data analysis, it is important to carry out this step carefully.

✓ <Practice Question 11-1-1>

1. Check the period of the data to see if it matches the description from UCI.
Implement it in Python and verify the results.
2. There is a column called `Country`. Count the number of records for each country.
3. Calculate the total sales amount for each country, create a ranking table, and then plot it as a bar graph.

Note: The answers are usually hidden right below the practice or comprehensive questions, but try to solve them without revealing the answers.

✓ Answer

```
print("From {0} to {1}".format(online_retail_data_table.InvoiceDate.min(), onli
```

```
From 2010-12-01 08:26:00 to 2011-12-09 12:50:00
```

```
online_retail_data_table.groupby(["Country"])["InvoiceNo"].count().sort_values(
```


InvoiceNo

The total sales purchase amount column will be added.

Country

online_retail_data_table['TotalPrice'] = online_retail_data_table.Quantity * or

Germany	9042
---------	------

```
rank_tp_df = online_retail_data_table.groupby(["Country"])["TotalPrice"].sum().rank_tp_df
```

Spain	2485
Netherlands	2363
Belgium	2031
Switzerland	1842
Portugal	1462
Australia	1185
Norway	1072
Italy	758
Channel Islands	748
Finland	685
Cyprus	614
Sweden	451
Austria	398
Denmark	380
Poland	330
Japan	321
Israel	248
Unspecified	244
Singapore	222
Iceland	182
USA	179
Canada	151
Greece	145
Malta	112
United Arab Emirates	68

European Community	60
RSA	58
Lebanon	45
Lithuania	35
Brazil	32
Czech Republic	25
Bahrain	17
Saudi Arabia	9

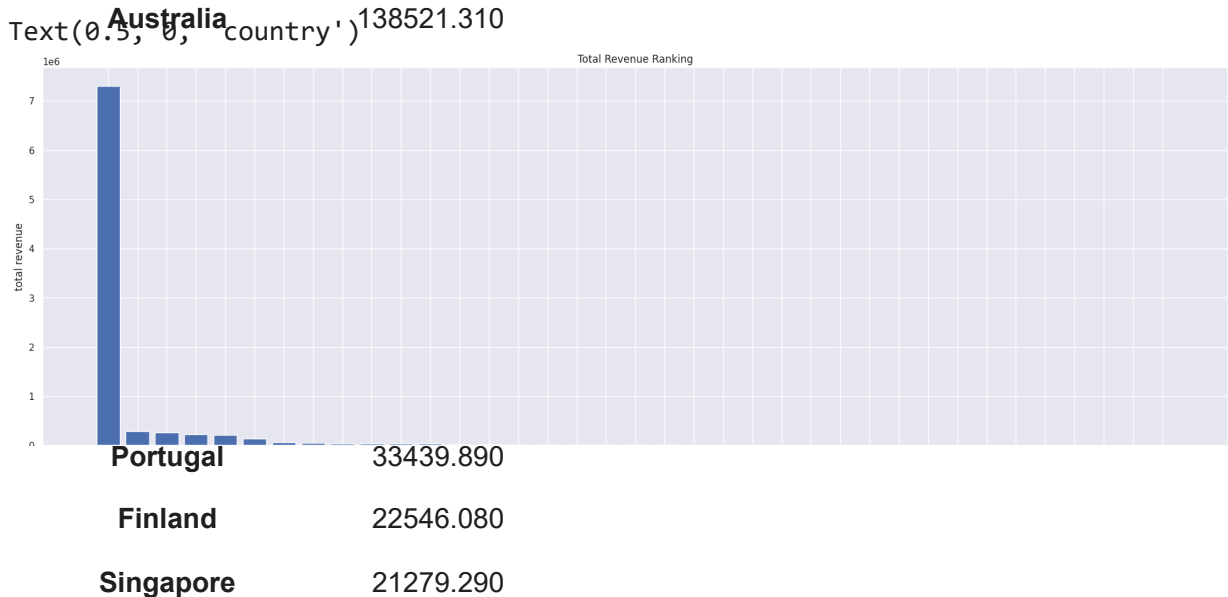
dtype: int64

TotalPrice

The bar chart displayed below shows the sales ranking by country. As expected, the UK has significantly higher sales.

United Kingdom 7308391.554

```
plt.figure(figsize=(25,8))
plt.xticks(rotation=90)
plt.bar(rank_tp_df.index, rank_tp_df,)
plt.title('Total Revenue Ranking')
plt.ylabel('total revenue')
plt.xlabel('country')
```



Side Note 4: Dealing with Uncertainty in Programming Implementation

Even after grasping the basics of programming and becoming somewhat proficient in coding, there may be times when you are unsure how to implement something, do not know how to write it, or forget the implementation method. In such cases, the first step is usually to "search for it." While reading books is important, there are situations where the information may not be available. Ideally, consulting official documentation is best, but that is most limited for intermediate or advanced users. Official documentation can be difficult to understand for beginners. Moreover, rather than giving up on the implementation after being unable to understand the documentation, it is crucial to narrow down your search keywords and keep looking for a solution to make some progress (of course, simply copying and pasting will not help you build real implementation skills).

Thus, "how to search" becomes important. It is better not to rely on just one article but to look at three or four different articles. This is because you cannot be sure if one method

is correct, and there may be better solutions available. It is also possible that there is a useful library you can use. To avoid reinventing the wheel, aim to implement efficiently.

European Community 1300.250

11-1.4 Basic Approaches to Data Exploration

Keywords: Basic Approaches to POS Data, Basic Aggregation, Fundamentals of Time Series Analysis

Czech Republic 826.740

Bahrain 548.400

11-1.4.1 Basic Data Aggregation and Checking

dtype: float64

Next, we will conduct basic aggregation and time series analysis on this POS data. In data analysis, there are typically two approaches: the hypothesis-driven approach mentioned initially, and the exploratory approach, which will be our focus in this section. While generating hypotheses is crucial, it can be challenging to do so without prior knowledge of the data. Therefore, in some cases, we may start by analyzing the data in an exploratory manner, advancing with the aim of gaining familiarity with it. However, as mentioned earlier, exploratory analysis can become endless, so it is recommended to work within a structured plan. Ideally, collaborating with someone knowledgeable in this business domain can be highly beneficial.

For example, it is essential to conduct basic data checks, such as determining the number of customers, the types and variety of products, and the volume of purchases in the data. By examining these results alongside the raw data, valuable insights can be gained. Additionally, while analyzing the data, issues in the data itself (potentially system-related issues) may be discovered. Although these steps may seem tedious, they can significantly impact the outcomes of the data analysis, so they should be carried out carefully.

First, let us check the volume of purchasing customers. Assuming that each `CustomerID` represents one customer (or one business entity), how many unique entities are present in this data? Here, we will use the `nunique` method from Pandas to count the unique values.

```
online_retail_data_table.CustomerID.nunique()
```

4339

From the result above, we can see that there are 4,339 unique business entities.

Next, let us determine how many unique types of products are present. We will count them based on the `StockCode` using the same method as before.

```
online_retail_data_table.StockCode.nunique()
```

```
3665
```

When examining the data by product name, we find that the count is slightly higher than the stock-based count. Why might this be? Could it simply be a system issue, or are other factors at play? While this may not directly impact customer actions, exploring potential causes as a practice in hypothesis testing will be addressed in a later exercise. (In practice, it is often faster to consult someone on-site who has a strong understanding of the business and can readily answer such questions.)

```
online_retail_data_table.Description.nunique()
```

```
3877
```

Next, let's calculate how much shopping has been done overall and the number of baskets (i.e., the number of receipts). Each unique `InvoiceNo` represents one receipt.

```
online_retail_data_table.InvoiceNo.nunique()
```

```
18536
```

From this, we can see that the total number of baskets per year is over 18,000.

<Practice Question 11-1-2>

1. Which customer has the most number of shopping transactions (baskets) in this data?
2. Which customer has the highest total purchase amount in this data?
3. Which product (based on `StockCode`) has the highest sales quantity in this data?
4. Which product (based on `StockCode`) has the highest total sales amount in this data?
5. In this data, the count of unique items based on product name is higher than the unique count based on `StockCode`. Consider a hypothesis for this discrepancy and implement (verify) it to confirm.

Answer

```
cust_invoice_rank_df = online_retail_data_table.groupby(['CustomerID'])['InvoiceNo']  
cust_invoice_rank_df.head(1)
```

	InvoiceNo
CustomerID	
12748	210

dtype: int64

```
cust_total_p_rank_df = online_retail_data_table.groupby(['CustomerID'])['TotalPrice']  
cust_total_p_rank_df.head(1)
```

	TotalPrice
CustomerID	
14646	280206.02

dtype: float64

```
stock_total_cnt_rank_df = online_retail_data_table.groupby(['StockCode'])['Quantity']  
stock_total_cnt_rank_df.head(1)
```

	Quantity
StockCode	
23843	80995

dtype: int64

```
stock_total_p_rank_df = online_retail_data_table.groupby(['StockCode'])['TotalPrice']  
stock_total_p_rank_df.head(1)
```

	TotalPrice
StockCode	
23843	168469.6

dtype: float64

There is an issue where the number of product codes does not match the number of product names. This means that even the same product code may have several different

product names associated with it. Consider this, assuming there is no error in the data.

The hypothesis could be that there was a change in product names over time, or perhaps product names differ by country. This is a plausible assumption but requires verification through hypothesis testing. Let's implement the following to verify:

The implementation below counts how many different product names exist for each `StockCode`. Normally, the count should be 1, but if multiple names exist for the same `StockCode`, the count will be 2 or more.

```
online_retail_data_table.groupby(["StockCode"])[ "Description"].nunique().sort_v
```

Description	
StockCode	
23196	4
23236	4
23203	3
17107D	3
23535	3
...	...
22427	1
22428	1
22429	1
22430	1
POST	1

3665 rows × 1 columns

dtype: int64

For example, let's focus on the `StockCode` 23236 at the top.

```
target_stock_df = online_retail_data_table[online_retail_data_table.StockCode==
target_stock_df
```

	InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice	Cus
218408	556042	23236	DOILEY STORAGE TIN	2	2011-06-08 12:44:00	2.89	
218586	556056	23236	DOILEY STORAGE TIN	1	2011-06-08 13:35:00	2.89	
219658	556106	23236	DOILEY STORAGE TIN	2	2011-06-08 16:54:00	2.89	
220367	556187	23236	DOILEY STORAGE TIN	6	2011-06-09 11:49:00	2.89	
220493	556195	23236	DOILEY STORAGE TIN	6	2011-06-09 12:21:00	2.89	
...	...	...	...	...	...	...	...
534004	581126	23236	STORAGE TIN VINTAGE DOILY	6	2011-12-07 12:43:00	2.89	
534688	581166	23236	STORAGE TIN VINTAGE DOILY	1	2011-12-07 14:43:00	2.89	
534700	581166	23236	STORAGE TIN VINTAGE DOILY	2	2011-12-07 14:43:00	2.89	
534738	581166	23236	STORAGE TIN VINTAGE DOILY	2	2011-12-07 14:43:00	2.89	
538984	581425	23236	STORAGE TIN VINTAGE DOILY	1	2011-12-08 15:31:00	2.89	

296 rows × 10 columns

Let's display all the raw data, as there are around 300 records in total. The following code changed on the Pandas settings to allow up to 300 rows to be displayed.

```
pd.set_option('display.max_rows', 300)
```

Looking at the data, it appears that from June to the first half of August 2011, the product was named "DOILEY STORAGE TIN," but after that, the name changed to "STORAGE TIN VINTAGE DOILY." It seems that there are no changes in the name based on the country. However, this is only a hypothesis, so if there are any information sources to verify this, it would be necessary to check them. By narrowing down the data with a hypothesis and examining the raw data, you can confirm such details. This kind of careful work should be done on a case-by-case basis.

```
target_stock_df
```


	InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice	Cus
218408	556042	23236	DOILEY STORAGE TIN	2	2011-06-08 12:44:00	2.89	
218586	556056	23236	DOILEY STORAGE TIN	1	2011-06-08	2.89	
219658	556106	23236	DOILEY STORAGE TIN	2	2011-06-08 16:54:00	2.89	

```
online_retail_data_table[online_retail_data_table.StockCode==23843]
```

	InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice	Cus
540421	581483	23843	PAPER CRAFT, LITTLE BIRDIE	80995	2011-12-09 09:15:00	2.08	
220700	580100	23236	BISCUIT TIN	1	2011-06-08 12:27:00	2.89	

Surprisingly, it was found in the data that one corporation purchased 80,995 units of this "PAPER CRAFT, LITTLE BIRDIE."

Now, most people will probably feel a sense of discomfort here. Looking only at the numbers, some might think, "Wow, they bought a lot," but from a practical, business perspective, it's more reasonable to ask, "Did this person place the order by mistake? Or was there some kind of issue with the system?"

It is important to consider that there might have been an issue with data preprocessing when performing data analysis. Often, in data analysis, we need to go back and review the data acquisition process. Finding such issues immediately is difficult, so it is important to be able to revisit the data when we notice something.

Earlier, the data before preprocessing was saved as tmp_online_retail_data_table. Let's use this to take another look at the records for StockCode 23843.

	InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice	Cus
223420	556484	23236	DOILEY STORAGE TIN	2	2011-06-12 13:17:00	2.89	
223975	556501	23236	DOILEY STORAGE TIN	6	2011-06-13 08:40:00	2.89	

InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice	Cus
223997	556502	DOILEY BISCUIT TIN	6	2011-06-13 08:43:00	2.89	
540421	581483	PAPER CRAFT , LITTLE BIRDIE	80995	2011-12-09 09:15:00	2.08	
224822	556578	DOILEY PAPER STORAGE	6	2011-06-13 14:12:00	2.89	
540422	C581484	PAPER CRAFT TIN LITTLE BIRDIE	-80995	2011-12-09 09:27:00	2.08	
225444	556680	DOILEY STORAGE TIN	6	2011-06-14 08:03:00	2.89	

From the two records above, it is clear that the purchase in the first record (InvoiceNo 581483) is canceled by the same amount in the second record (InvoiceNo C581484). If InvoiceNo 581484 is a cancellation for InvoiceNo 581483, then the preprocessing method previously implemented is fine. However, if 581483 is a cancellation for 581484, then 581483 was in the correct order and should also be deleted.

Let's take a quick look at the cancelled data.

InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice	Cus
225700	556728	DOILEY STORAGE TIN	3	2011-06-14 11:36:00	2.89	
tmp_online_retail_data_table[tmp_online_retail_data_table.cancel_flg=="C"]						
225720	556731	STORAGE TIN	1	2011-06-14 11:41:00	2.89	
225915	556751	DOILEY STORAGE TIN	48	2011-06-14 12:09:00	2.49	
226345	556790	DOILEY STORAGE TIN	6	2011-06-14 13:48:00	2.89	
227349	556845	DOILEY STORAGE TIN	48	2011-06-15 11:30:00	2.49	
227670	556891	DOILEY BISCUIT TIN	6	2011-06-15 12:48:00	2.89	
228180	556917	DOILEY STORAGE TIN	144	2011-06-15 13:37:00	2.49	
228403	556928	DOILEY BISCUIT TIN	48	2011-06-15 14:48:00	2.49	
228883	556964	DOILEY STORAGE TIN	6	2011-06-16 09:46:00	2.89	

229104	InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice	Cus
141	C536379	D	Discount	-1	2010-12-01 09:41:00	27.50	
-----	-----	-----	-----	-	12:05:00	----	
154	C536383	35004C	COLOURED DOILEY FLYING DUCKS TIN	-1	2010-12-01 09:49:00	4.65	
229411	557040	23236	DOILEY STORAGE TINS	6	2011-06-16 14:22:00	2.89	
235	C536391	22556	PLASTERS IN TIN CIRCUS PARADE	-12	2010-12-01 10:24:00	1.65	
229590	557064	23236	PACK OF 12 DOILEY STORAGE TINS	2	2011-06-16 19:07:00	2.89	
236	C536391	21984	PAISLEY TISSUES DOILEY	-24	2010-12-01 10:24:00	0.29	
237	C536391	21983	PACK OF 12 BLUE PAISLEY TISSUES	-24	2010-12-01 10:24:00	0.29	
...	...	...	DOILEY ZINC T-LIGHT HOLDER STARS SMALL	...	2011-06-16	...	
540449	C581490	23144	DOILEY BISCUIT TIN	-11	2011-12-09 09:57:00	0.83	
541541	C581499	M	VICTORIAN SEWING BOX LARGE	-1	2011-12-09 11:57:00	224.69	
232738	557327	23236	HANGING DOILEY HEART JAR STORAGE T-LIGHT TIN	6	2011-06-20 10:28:00	2.89	
541715	C581568	21258	DOILEY STORAGE TINS	-5	2011-12-09 11:57:00	10.95	
233098	C581569	23236	36 PENCILS TUBE RED RETROSPOT	12	2011-12-09 11:58:00	2.25	
541717	C581569	20979	DOILEY STORAGE TINS	-5	2011-12-09 11:58:00	1.25	
8905 rows × 9 columns	557484	23236	DOILEY STORAGE TINS	1	2011-06-20 14:36:00	2.89	
233760	557500	23236	DOILEY STORAGE TINS	1	2011-06-20 15:47:00	2.89	

Let's pick up a few customers and look at the data. First, let's look at the person with CustomerID 17548.

DOILEY
2011-06-22

```
tmp_online_retail_data_table[(tmp_online_retail_data_table.CustomerID==17548)]
```

236905	557789	23236	DOILEY STORAGE TIN	6	2011-06-22 16:32:00	2.89
237110	557801	23236	DOILEY STORAGE TIN	6	2011-06-23 10:14:00	2.89
237280	557850	23236	DOILEY STORAGE TIN	6	2011-06-23 10:56:00	2.89
237633	557878	23236	DOILEY STORAGE TIN	4	2011-06-23 13:13:00	2.89
238870	557957	23236	DOILEY STORAGE TIN	3	2011-06-23 19:20:00	2.89
240034	558084	23236	DOILEY STORAGE TIN	6	2011-06-26 11:47:00	2.89
241578	558235	23236	DOILEY STORAGE TIN	6	2011-06-27 14:41:00	2.89
242011	558271	23236	DOILEY STORAGE TIN	6	2011-06-28 10:17:00	2.89
242578	558358	23236	DOILEY STORAGE TIN	1	2011-06-28 15:05:00	2.89
242896	558375	23236	DOILEY STORAGE TIN	6	2011-06-28 16:02:00	2.89
243121	558419	23236	DOILEY STORAGE TIN	6	2011-06-29 11:41:00	2.89
244896	558558	23236	DOILEY STORAGE TIN	1	2011-06-30 13:17:00	2.89
246358	558699	23236	DOILEY STORAGE TIN	6	2011-07-01 12:17:00	2.89
248579	558861	23236	DOILEY STORAGE TIN	6	2011-07-04 12:19:00	2.89
			DOILEY		2011-07-04	

249088	558881	23236	STORAGE TIN	6	14:36:00	2.89
249175	558888	23236	DOILEY STORAGE TIN	1	2011-07-04 15:29:00	2.89
249830	558920	23236	DOILEY BISCUIT TIN	6	2011-07-05 09:57:00	2.89
250135	559004	23236	DOILEY STORAGE TIN	12	2011-07-05 12:23:00	2.89
252756	559134	23236	DOILEY STORAGE TIN	6	2011-07-06 13:14:00	2.89
252931	559145	23236	DOILEY STORAGE TIN	6	2011-07-06 14:17:00	2.89
253041	559156	23236	DOILEY STORAGE TIN	6	2011-07-06 15:50:00	2.89
253049	559157	23236	DOILEY STORAGE TIN	6	2011-07-06 15:53:00	2.89
253695	559168	23236	DOILEY STORAGE TIN	2	2011-07-06 17:05:00	2.89
253843	559199	23236	DOILEY STORAGE TIN	12	2011-07-07 10:28:00	2.89
253861	559200	23236	DOILEY STORAGE TIN	3	2011-07-07 10:36:00	2.89
254215	559300	23236	DOILEY STORAGE TIN	6	2011-07-07 12:40:00	2.89
258308	559607	23236	DOILEY STORAGE TIN	6	2011-07-11 11:39:00	2.89
258984	559679	23236	DOILEY STORAGE TIN	6	2011-07-11 14:46:00	2.89
259816	559733	23236	DOILEY STORAGE TIN	6	2011-07-12 10:54:00	2.89
260468	559813	23236	DOILEY STORAGE TIN	6	2011-07-12	2.89

Looking at the data above, there are cancelation records right away. However, since the data only started from December 2010, the items purchased before that might have been canceled. Also, there were some purchases made on April 20, 2011, and there are cancelation records for several items on May 6, 2011. When comparing with the purchase data from April 20, they match exactly. This suggests that the cancelation record was not for InvoiceNo 552049, but for a prior purchase (InvoiceNo 550755) that was canceled.

When looking at other similar records, the data structure is the same, and it becomes clear that the initial preprocessing might not have been completely appropriate. While it's possible to implement a process to clear these cancelation records, it would be more difficult than the previous preprocessing. This is why data preprocessing is said to make up 80% of the work. People who do not engage in data analysis may not understand the difficulty of data preprocessing since it lacks fun and consumes a lot of time. However, there is no choice but to steadily work through it. AI will not solve these kinds of problems. This is also a system issue, and it may be necessary to investigate the cause and request changes to the logging specifications. While it would be ideal to have a program that can easily process it, the problem arises when the key that links the order to its cancelation is missing in the system logs. The first step should be to find the root cause of the issue in the system logs, and there is no need to force a solution.

In this lecture, the focus is on learning the basic methods for consumer analysis, not on data preprocessing in depth. Since the number of records is under 9,000 out of the 400,000 total, it may feel a bit awkward, but we will continue with the initial preprocessing for now. However, in practical terms, handling canceled orders is necessary, and how to address this will be covered in the final comprehensive problem.

Now, let's temporarily move away from the initial data preprocessing and reconsider it from the perspective of sales rankings. Looking at the second-place item, "REGENCY CAKESTAND 3 TIER" (StockCode 22423), we can see the following records.

InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice
165028	550755	IVORY GIANT GARDEN	4	2011-04-20	5.95
online_retail_data_table[online_retail_data_table.StockCode==22423]					
177223	C552049	IVORY GIANT GARDEN THERMOMETER	-4	2011-05-06	5.95
177224	C552049	PACK OF 6 DOLEY BIRDY GIFT TAGS	-24	2011-05-06	1.25
275321	560977	STORAGE TINS	6	2011-07-22	2.89
177225	C552049	RIBBON REEL STRIPES	10	2011-05-06	1.65

InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice	Cus
880	536477	REGENCY CAKESTAND 3 TIER	16	2010-12-01 12:27:00	10.95	
177227	C552049	HEARTS REGENCY DESIGN	-10	2011-05-01 10:00:00	1.65	
936	536502	REGENCY CAKESTAND 3 TIER	2	2010-12-01 12:36:00	12.75	
1092	536525	REGENCY CAKESTAND 3 TIER	2	2010-12-01 12:54:00	12.75	
217861	536528	REGENCY CAKESTAND 3 TIER	6	2010-07-25 13:54:00	12.85	
1197	536530	REGENCY CAKESTAND 3 TIER	1	2010-12-01 13:21:00	12.75	
...	...	DOILEY STORAGE TIN	...	2011-07-27	...	...
538944	581425	REGENCY CAKESTAND 3 TIER	1	2011-12-08 15:31:00	12.75	
281959	581614	STORAGE REGENCY TIN	6	12:50:00	2.89	
539891	581449	REGENCY CAKESTAND 3 TIER	1	2011-12-08 17:37:00	12.75	
...	...	DOILEY STORAGE TIN	...	2011-07-28	...	...
539892	581449	REGENCY CAKESTAND 3 TIER	1	2011-12-08 17:37:00	12.75	
282316	581642	STORAGE REGENCY TIN	2	15:04:00	2.89	
540216	581472	REGENCY CAKESTAND 3 TIER	2	2011-12-08 19:55:00	12.75	
...	...	DOILEY STORAGE TIN	...	2011-07-28	...	...
541231	581495	REGENCY CAKESTAND 3 TIER	10	2011-12-09 10:20:00	12.75	
283756	581801	STORAGE REGENCY TIN	12	14:14:00	2.89	
1724 rows x 10 columns						
284944	561893	DOILEY STORAGE TIN	1	2011-07-31 14:39:00	2.89	

Unlike the number one item, "PAPER CRAFT, LITTLE BIRDIE," "REGENCY CAKESTAND 3 TIER" has 1,724 records, and from the following summary results, we can see that it was purchased by 881 different companies. Therefore, it is clear that this product "REGENCY CAKESTAND 3 TIER" is being purchased by a wider range of companies.

285385	561902	DOILEY STORAGE TIN	48	2011-07-31 15:50:00	2.89	
285861	561926	DOILEY STORAGE TIN	6	2011-08-01	2.89	

```
online_retail_data_table[online_retail_data_table.StockCode==22423].CustomerID.
```

8286663	562034	23236	DOILEY STORAGE TIN	6	2011-08-02 09:14:00	2.89
---------	--------	-------	--------------------	---	---------------------	------

Furthermore, looking at the data above, it can be seen that the UnitPrice for the same product varies, which suggests that a discount or some other adjustment might have been applied. What to do with this data depends on the objective, but it could serve as material for considering optimal pricing strategies. However, since pricing falls outside the scope of the current chapter (which focuses on customer identification), it will be briefly introduced in the Advanced Consumer Analysis section.

288967	562214	23236	DOILEY STORAGE TIN	6	2011-08-03 13:30:00	2.89
--------	--------	-------	--------------------	---	---------------------	------

```
online_retail_data_table[online_retail_data_table.StockCode==22423].UnitPrice.v
```

289515	562269	23236	DOILEY STORAGE TIN	2	2011-08-03 16:35:00	2.89
UnitPrice						
12.75	1401	23236	DOILEY STORAGE TIN	1	2011-08-04 12:22:00	2.89
10.95	309					
24.96	10	23236	DOILEY STORAGE TIN	6	2011-08-04 17:58:00	2.89
4.00	2					
0.00	1	23236	DOILEY STORAGE TIN	6	2011-08-05 11:46:00	2.89
5.00	1					
292260	562543	23236	DOILEY STORAGE TIN	6	2011-08-05 15:45:00	2.89

As a repeated point, by looking at each piece of data in such situations, there are things that cannot be obtained from simple aggregation results or analysis alone, so make sure not to forget to check the raw data as needed.

✓ Side Note 6: What indicators should be used to compare product sales?

The analysis objective here is to consider whether to change the products offered in each country. For example, let's focus on the product with StockCode 22423. Can we say that this product sells better in the UK or Germany? (While this may not be a typical perspective for an e-commerce site, it is a common approach when comparing in-store purchases between different locations.)

In terms of total sales, it is expected that the UK will have higher sales. However, does that necessarily mean that the product is more popular in the UK? Naturally, more people shop in the UK, so higher sales across all products is expected.

In analysis, there is a principle called “apples to apples,” meaning comparisons should be made between like-for-like metrics. To illustrate this, we introduce the Purchase Incidence (PI) metric, which is commonly used in retail marketing analysis. PI compares the sales of a target product between stores with different customer footfalls. Strictly speaking, customer footfall is used in the denominator. However, in this case, we will treat the total number of baskets as a proxy for the number of customers and calculate accordingly using POS data from the e-commerce site.

The following shows the number of baskets for each country. As expected, the UK has the highest number of purchases.

```
country_incnt_rank_df = online_retail_data_table.groupby(["Country"])[ "InvoiceN
country_incnt_rank_df
```

			TIN			09:14:00	
			STORAGE				
299557	563110	23236	TIN	6	2011-08-12	2.89	
			VINTAGE		11:07:00		
			DOILY				
			STORAGE				
302762	563444	23236	TIN	1	2011-08-16	2.89	
			VINTAGE		13:50:00		
			DOILY				
			STORAGE				
302900	563477	23236	TIN	1	2011-08-16	2.89	
			VINTAGE		15:39:00		
			DOILY				
			STORAGE				
304331	563569	23236	TIN	6	2011-08-17	2.89	
			VINTAGE		14:47:00		
			DOILY				
			STORAGE				
304410	563574	23236	TIN	6	2011-08-17	2.89	
			VINTAGE		15:05:00		
			DOILY				
			DOILEY				
304798	563613	23236	STORAGE	12	2011-08-18	2.89	
			TIN		08:49:00		
			STORAGE				
305319	563680	23236	TIN	6	2011-08-18	2.89	
			VINTAGE		12:06:00		
			DOILY				
			STORAGE				
307587	563924	23236	TIN	2	2011-08-21	2.89	
			VINTAGE		14:22:00		

			VINTAGE DOILY		14:02:00	
307756	563927	23236	STORAGE TIN VINTAGE DOILY	6	2011-08-21 15:01:00	2.89
308304	563947	23236	STORAGE TIN VINTAGE DOILY	24	2011-08-22 10:18:00	2.89
308337	563950	23236	STORAGE TIN VINTAGE DOILY	6	2011-08-22 10:39:00	2.89
309465	564090	23236	STORAGE TIN VINTAGE DOILY	6	2011-08-23 09:53:00	2.89
309582	564095	23236	DOILEY BISCUIT TIN	6	2011-08-23 10:31:00	2.89
310049	564162	23236	STORAGE TIN VINTAGE DOILY	6	2011-08-23 13:27:00	2.89
310586	564190	23236	STORAGE TIN VINTAGE DOILY	6	2011-08-23 16:01:00	2.89
310761	564199	23236	STORAGE TIN VINTAGE DOILY	1	2011-08-23 17:31:00	2.89
310778	564199	23236	STORAGE TIN VINTAGE DOILY	1	2011-08-23 17:31:00	2.89
311331	564256	23236	STORAGE TIN VINTAGE DOILY	1	2011-08-24 10:45:00	2.89
311497	564301	23236	STORAGE TIN VINTAGE DOILY	6	2011-08-24 11:54:00	2.89
311589	564305	23236	STORAGE TIN VINTAGE	6	2011-08-24 12:20:00	2.89

DOILY

InvoiceNo

Next, the total sales amount for product code 22423 in each country is calculated. As anticipated, the UK ranks first.

Country	InvoiceNo	Product Code	Quantity	Unit Price	Total Price
United Kingdom	16649	22423	6	2.89	17.28

```
country_total_rank_df = online_retail_data_table[online_retail_data_table.Stock
country_total_rank_df
```

Country	InvoiceNo	Product Code	Quantity	Unit Price	Total Price
EIRE	260	22423	6	2.89	17.28
Belgium	98	22423	6	2.89	17.28
Netherlands	95	22423	6	2.89	17.28
Spain	90	22423	6	2.89	17.28
Australia	57	22423	6	2.89	17.28
Portugal	57	22423	6	2.89	17.28
Switzerland	51	22423	6	2.89	17.28
Finland	41	22423	6	2.89	17.28
Italy	38	22423	6	2.89	17.28
Norway	36	22423	1	2.89	2.89
Sweden	36	22423	6	2.89	17.28
Channel Islands	26	22423	6	2.89	17.28
Poland	19	22423	6	2.89	17.28
Japan	19	22423	6	2.89	17.28
Denmark	18	22423	6	2.89	17.28
Austria	17	22423	6	2.89	17.28
Cyprus	16	22423	6	2.89	17.28
Unspecified	8	22423	1	2.89	2.89
Iceland	7	22423	6	2.89	17.28
Singapore	7	22423	6	2.89	17.28
Canada	6	22423	6	2.89	17.28
Malta	5	22423	6	2.89	17.28
Israel	5	22423	4	2.89	11.56
Greece	5	22423	4	2.89	11.56
USA	5	22423	4	2.89	11.56
Lithuania	4	22423	2	2.89	5.78

European Community	4	VINTAGE DOILY	12:41:00	
United Arab Emirates	3	STORAGE		
Czech Republic	2	TIN	2011-09-05	2.49
326467	23236	VINTAGE DOILY	10:47:00	
Bahrain	2	DOILEY		
Lebanon	1	STORAGE	2011-09-05	2.89
326252	23236	TIN	14:30:00	
RSA	1	STORAGE		
Saudi Arabia	1	TIN	2011-09-06	2.89
Brazil	1	VINTAGE DOILY	10:42:00	
dtype: int64		STORAGE		
327617	565682	TIN	2011-09-06	2.89
	23236	VINTAGE DOILY	10:42:00	
		STORAGE		
328700	565801	TIN	2011-09-07	2.89
	23236	VINTAGE DOILY	10:28:00	
		STORAGE		
331502	566018	TIN	2011-09-08	2.89
	23236	VINTAGE DOILY	12:42:00	
		STORAGE		
331527	566018	TIN	2011-09-08	2.89
	23236	VINTAGE DOILY	12:42:00	
		STORAGE		
331878	566041	TIN	2011-09-08	2.89
	23236	VINTAGE DOILY	13:48:00	
		STORAGE		
332497	566076	TIN	2011-09-09	2.89
	23236	VINTAGE DOILY	09:13:00	
		STORAGE		
333284	566191	TIN	2011-09-09	2.89
	23236	VINTAGE DOILY	13:26:00	
		STORAGE		
333466	566195	TIN	2011-09-09	2.89
	23236	VINTAGE DOILY	13:44:00	
		STORAGE		
333726	566212	TIN	2011-09-09	2.89

333720	300213	23230	VINTAGE	6	15:09:00	2.89
TotalPrice DOILY						
Country STORAGE						
United Kingdom TIN						
110990.20			VINTAGE	6	2011-09-11	2.89
Germany DOILY						
9061.95					14:15:00	

```
merge_country_df = pd.concat([country_incnt_rank_df, country_total_rank_df]
                              , axis=1
                              , join='inner')
merge_country_df.columns = ['cnt', 'totalP']
merge_country_df
```

336201	Spain	566308	2049.00	TIN	6	2011-09-12	2.89
			23230	VINTAGE		09:38:00	
	Australia		1978.20	DOILY			
	Switzerland		924.15	STORAGE			
	Belgium		599.25	TIN	6	2011-09-12	2.89
				VINTAGE		11:25:00	
	Israel		551.10	DOILY			
	Channel Islands		517.80	STORAGE			
				TIN	6	2011-09-12	2.89
330303	Cyprus	300303	23230	VINTAGE		12:45:00	
			382.50	DOILY			
	Norway		357.00	STORAGE			
337109	Portugal	566427	23230	TIN	2	2011-09-12	2.89
				VINTAGE		14:24:00	
	Iceland		191.25	DOILY			
	Brazil		175.20	STORAGE			
	Greece		175.20	TIN	144	2011-09-13	2.89
				VINTAGE		11:12:00	
	Singapore		175.20	DOILY			
	Lebanon		153.00	STORAGE			
330017	United Arab Emirates	300017	23230	TIN	6	2011-09-13	2.89
				VINTAGE		12:13:00	
				DOILY			
	USA		114.75	STORAGE			
340111	Malta	566605	23230	TIN	2	2011-09-13	2.89
			89.25	VINTAGE		16:27:00	
	Italy		76.50	DOILY			
	Austria		51.00	STORAGE			
	Finland		38.25	TIN	6	2011-09-14	2.89
				VINTAGE		09:27:00	
	RSA		25.50	DOILY			
	Bahrain		25.50	DOILEY	6	2011-09-14	2.89
				ISCUIT TIN		10:34:00	
	Unspecified		25.50	STORAGE			
	Canada		12.75	TIN	6	2011-09-14	2.89
				VINTAGE		11:53:00	

dtype: float64			DOILY			
341071	566733	23236	STORAGE TIN VINTAGE DOILY	6	2011-09-14 13:54:00	2.89
341338	566756	23236	STORAGE TIN VINTAGE DOILY	6	2011-09-14 15:54:00	2.89
344579	567085	23236	STORAGE TIN VINTAGE DOILY	6	2011-09-16 12:38:00	2.89
344649	567092	23236	STORAGE TIN VINTAGE DOILY	6	2011-09-16 12:48:00	2.89
346086	567183	23236	STORAGE TIN VINTAGE DOILY	2	2011-09-18 15:32:00	2.89
349366	567504	23236	DOILEY STORAGE TIN	6	2011-09-20 14:33:00	2.89
350013	567610	23236	STORAGE TIN VINTAGE DOILY	48	2011-09-21 11:30:00	2.49
350848	567656	23236	STORAGE TIN VINTAGE DOILY	1	2011-09-21 14:40:00	5.79
353851	567850	23236	STORAGE TIN VINTAGE DOILY	6	2011-09-22 13:18:00	2.89
354285	567873	23236	STORAGE TIN VINTAGE DOILY	6	2011-09-22 14:25:00	2.89
356504	568056	23236	STORAGE TIN VINTAGE DOILY	12	2011-09-23 12:55:00	2.89
358407	568145	23236	STORAGE TIN VINTAGE	18	2011-09-25 11:30:00	2.89

Since we have the purchase amount and basket count for each country from the data above, the "PI value" (or a similar metric) is calculated as follows, and then sorted for ranking. (Since the PI value is the purchase index per 1000 customers passing through the register, it is multiplied by 1000.)

```
merge_country_df['PI'] = merge_country_df.totalP / merge_country_df.cnt * 1000
merge_country_df.PI.sort_values(ascending=False)
```

Belgium	98	599.25					
364090 Netherlands	23236	3166.35	6	2011-09-28	11:56:00	2.89	
Spain	90	2049.00					
364120 Australia	57	1978.20	6	2011-09-28	12:28:00	2.89	
Portugal	57	328.20					
Switzerland	51	924.15					
Finland	41	38.25	6	2011-09-28	13:43:00	2.89	
Italy	38	76.50					
Norway	36	357.00					
365224 Channel Islands	23236	517.80	6	2011-09-28	15:07:00	2.89	
Austria	17	51.00					
366604 Cyprus	16	382.50	1	2011-09-29	09:23:00	2.89	
Unspecified	8	25.50					
Iceland	7	191.25					
Singapore	7	175.20	6	2011-09-29	12:00:00	2.89	
Canada	6	12.75					
Malta	5	89.25					
367805 Israel	23236	551.10	6	2011-09-29	13:38:00	2.89	
Greece	5	175.20					
368220 USA	5	114.75	6	2011-09-30	10:18:00	2.89	
United Arab Emirates	3	153.00					
Bahrain	2	25.50					
Lebanon	1	153.00	6	2011-09-30	10:19:00	2.89	
RSA	1	25.50					
Brazil	1	175.20					

369544	569037	23236	TIN VINTAGE DOILY	2	2011-09-30 10:45:00	2.89
370641	569147	23236	STORAGE TIN VINTAGE DOILY	6	2011-09-30 15:33:00	2.89
372453	569243	23236	STORAGE TIN VINTAGE DOILY	6	2011-10-03 09:44:00	2.89
375497	569468	23236	STORAGE TIN VINTAGE DOILY	6	2011-10-04 11:56:00	2.89
376015	569491	23236	STORAGE TIN VINTAGE DOILY	3	2011-10-04 13:25:00	2.89
376140	569496	23236	STORAGE TIN VINTAGE DOILY	6	2011-10-04 13:45:00	2.89
376205	569518	23236	STORAGE TIN VINTAGE DOILY	3	2011-10-04 14:28:00	2.89
376478	569524	23236	STORAGE TIN VINTAGE DOILY	2	2011-10-04 14:42:00	2.89
377522	569555	23236	STORAGE TIN VINTAGE DOILY	6	2011-10-05 08:17:00	2.89
377723	569563	23236	STORAGE TIN VINTAGE DOILY	12	2011-10-05 09:46:00	2.89
378126	569630	23236	STORAGE TIN VINTAGE DOILY	6	2011-10-05 11:44:00	2.89
378414	569650	23236	STORAGE TIN VINTAGE DOILY	48	2011-10-05 12:44:00	2.49

From the results above, the PI value for this product is 6666 in the United Kingdom and 19829 in Germany, indicating that it sells better in Germany.

Additionally, the insights that can be drawn from this data include the fact that the product is particularly purchased in Brazil, although the total number of baskets is low, so this should be taken as a reference. Finland has the lowest PI value for this product. "REGENCY CAKESTAND 3 TIER" is a product that seems to be commonly used for afternoon tea, based on the image. This could be because in Finland, coffee is more popular than tea, and afternoon tea was still uncommon in the early 2010s. This cultural difference might explain the low PI value for this product in Finland. However, this is merely a hypothesis, so please consider it as reference information.

Reference: Rankings can also be assigned using the rank method. Since this will not be used in later processing, the implementation is omitted.

<Practice Question 11-1-3>

1. Let's calculate the PI value for other products

(Answer omitted)

Channel Islands

11-1.4.2 Basic Time Series Analysis

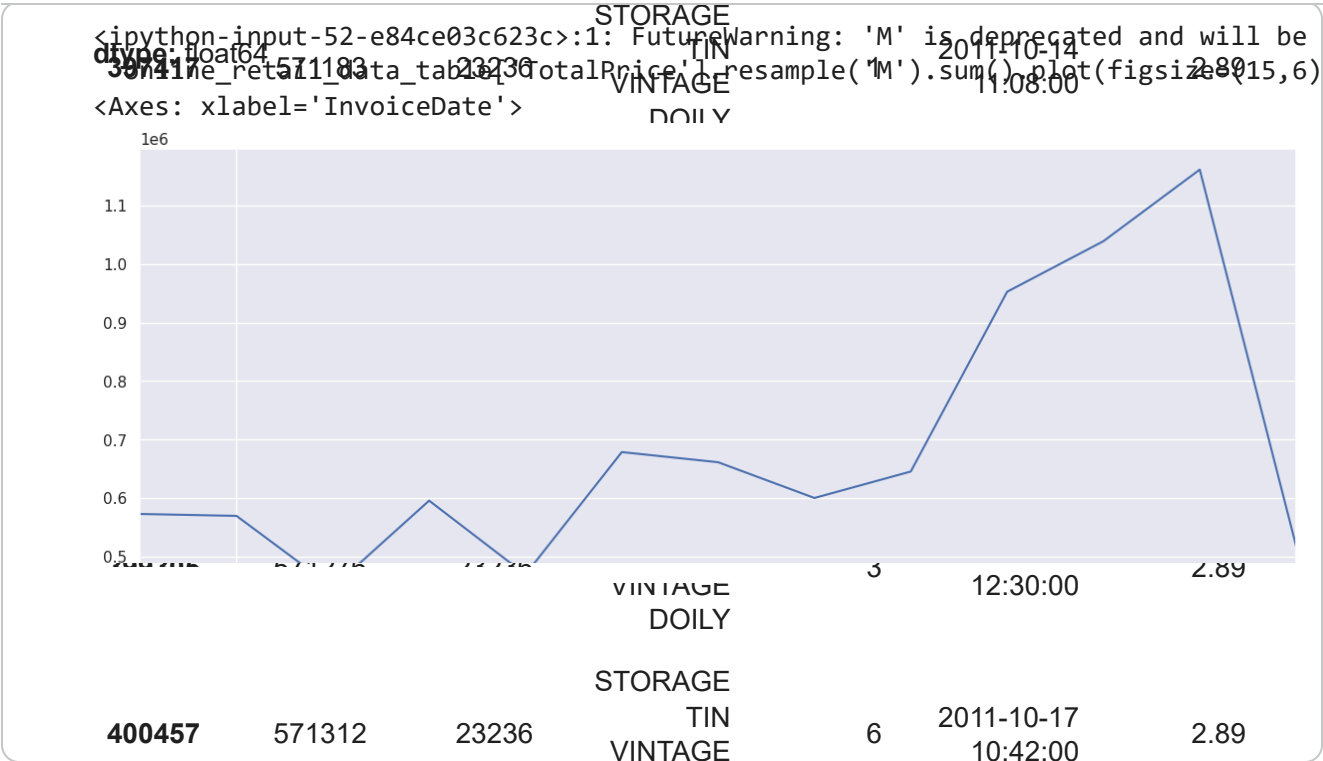
Until now, we have been discussing the total amount of sales for the entire dataset. However, basic aggregation alone doesn't show trends like whether sales are going up or down, as it is only a snapshot in time. Therefore, the next goal is to understand the sales trend over time. To do this, we will generate time series graphs of sales on a monthly or daily basis.

To make time series data using the Pandas framework, we will set the date and time as the index.

```
online_retail_data_table.set_index(['InvoiceDate'], inplace=True)
```

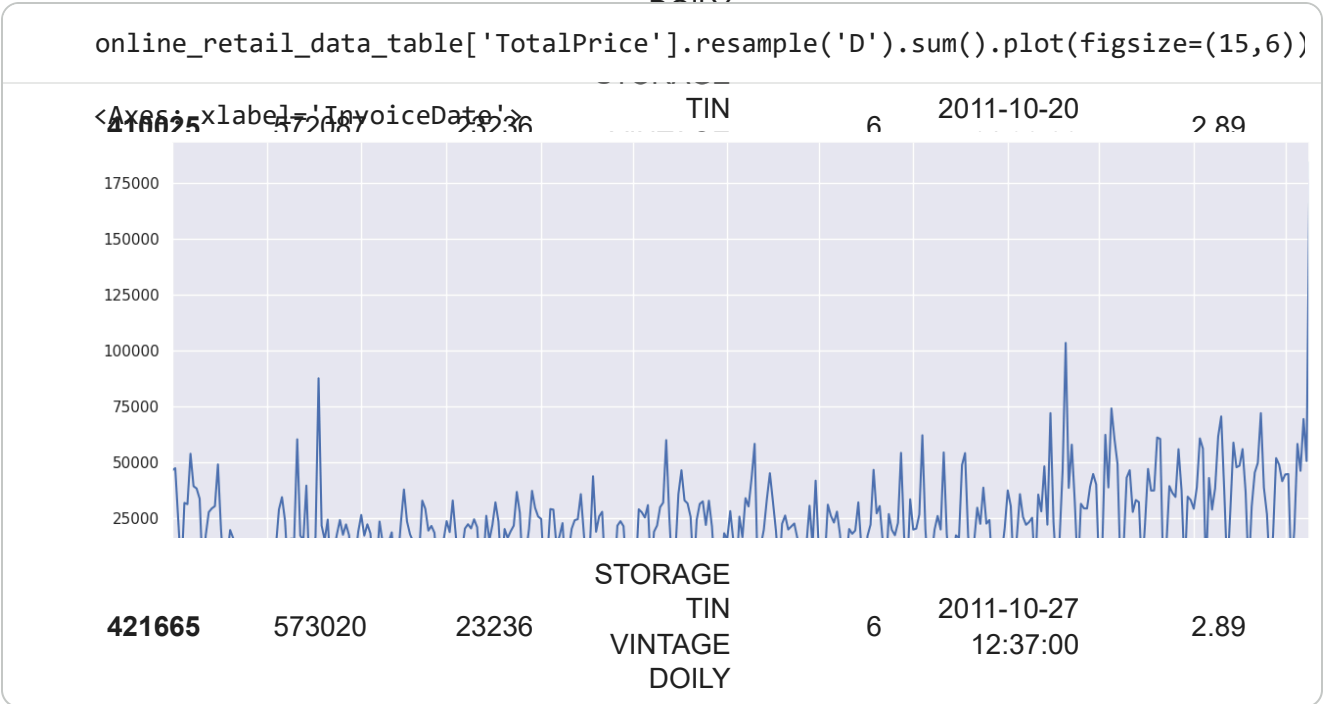
The following uses the Pandas resample method to calculate the sales amount on a monthly basis. The parameter 'M' specifies the frequency. This also serves as a review of Pandas.

```
online_retail_data_table['TotalPrice'].resample('M').sum().plot(figsize=(15,6))
```



Looking at this, it seems that sales gradually increased from the starting point, with a peak in November 2011. In addition, it should be noted that sales in December 2011 are not necessarily lower; this is simply because the period only covers part of the month, so we cannot conclude that sales decreased in December based on this data alone.

Next, let's look at the daily sales.



Looking at this, we can see that there seems to be some periodicity in the sales. Additionally, there are spikes in sales in mid-January 2011, late September 2011, and

early December 2011. It's unclear from the data whether these spikes were caused by a campaign or due to certain customers, but further investigation, such as examining sales rankings during these periods or performing a sales decomposition analysis, could help clarify the reasons. As observed earlier in the raw data, the spike in early December 2011 is likely due to erroneous orders being included.

This concludes the introduction to basic aggregations and observing time-series trends using this POS data.

In exploratory analysis, statistical approaches and descriptive statistics can also be employed to calculate metrics such as the mean and variance of sales and quantities for each product, or to create box plots and histograms. However, as this chapter is not a lecture on descriptive statistics, these aspects are omitted. For those who feel uncertain about these topics, even though the target data may differ, it is recommended to review materials from week 4 as well as other textbooks.

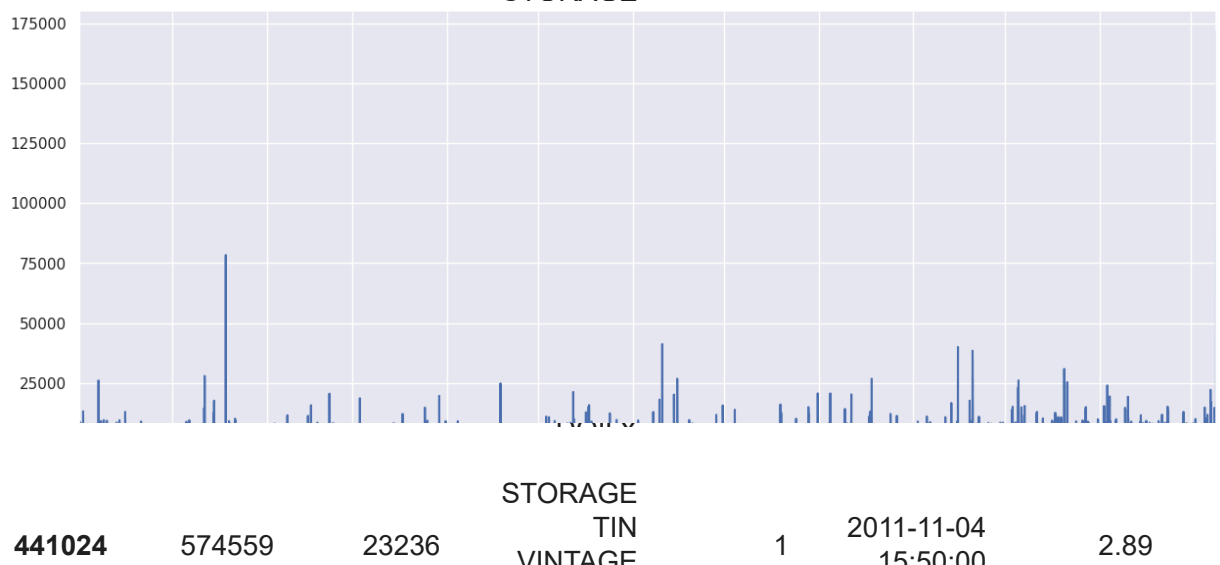
<Practice Question 11-1-4>

1. Calculate the total sales by hour from the data above and plot it on a graph.
2. Furthermore, identify the hour with the highest sales and investigate what product sold the most and which customer purchased it.

✓ Answer

```
online_retail_data_table['TotalPrice'].resample('H').sum().plot(figsize=(15,6))
```

```
<ipython-input-54-94bee57a7ffd>:1: FutureWarning: 'H' is deprecated and will be
online_retail_data_table['TotalPrice'].resample('H').sum().plot(figsize=(15,6))
<Axes: xlabel='InvoiceDate'>
```



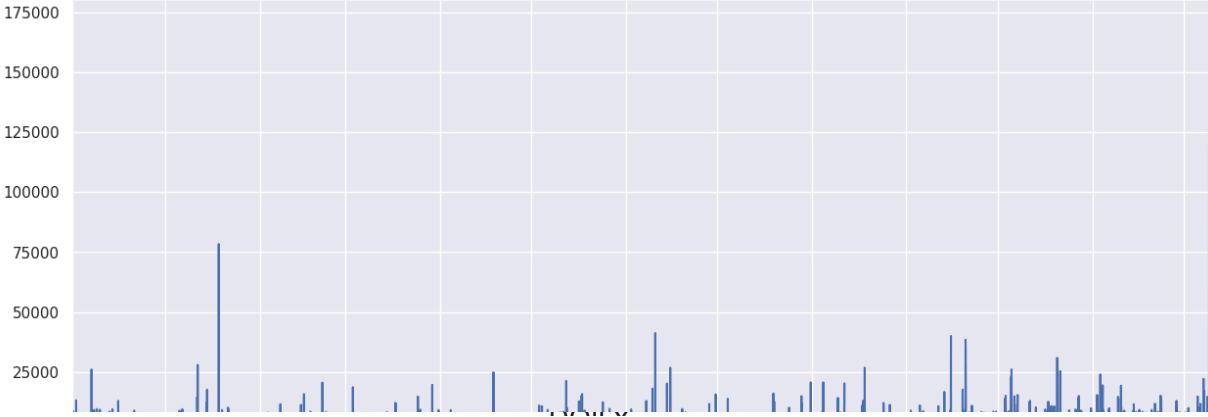
DOILY

```
online_retail_data_table['TotalPrice'].resample('H').sum().plot(figsize=(15,6))
```

<ipython-input-55-94bee57a7ffd>:1: FutureWarning: 'H' is deprecated and will be removed in a future version. Please use 'h' instead.

```
online_retail_data_table['TotalPrice'].resample('H').sum().plot(figsize=(15,6))
```

<Axes: xlabel='InvoiceDate'>



442795 574682 23236 STORAGE TIN 1 2011-11-06 12:44:00 2.89 DOILY

DOILY

```
online_retail_data_table['TotalPrice'].resample('H').sum()[online_retail_data_t
```

<ipython-input-56-416947bc8777>:1: FutureWarning: 'H' is deprecated and will be removed in a future version. Please use 'h' instead.

```
online_retail_data_table['TotalPrice'].resample('H').sum()[online_retail_data_t
```

TotalPrice	InvoiceDate	STORAGE	TIN			
445856	574863	23236	1	2011-11-07	12:29:00	2.89
2011-12-09 09:00:00	171679.08					
445856	574863	23236	1	2011-11-07	12:29:00	2.89

DOILY

The following data is filtered to focus on December 9, 2011, at 9 a.m. Pandas is useful since it is able to do this type of filtering.

DOILY

```
online_retail_data_table.loc["2011-12-09 09"]
```

455169	575607	23236	3	2011-11-10	12:37:00	5.79
--------	--------	-------	---	------------	----------	------

DOILY

455640	575634	23236	TIN VINTAGE DOILY	6	2011-11-10 13:44:00	2.89
455884	575654	23236	STORAGE TIN VINTAGE DOILY	6	2011-11-10 14:28:00	2.89
456555	575697	23236	DOILEY STORAGE TIN	6	2011-11-10 16:51:00	2.89
458475	575840	23236	STORAGE TIN VINTAGE DOILY	1	2011-11-11 11:56:00	2.89
460139	575896	23236	STORAGE TIN VINTAGE DOILY	6	2011-11-11 14:52:00	2.89
460499	575918	23236	STORAGE TIN VINTAGE DOILY	3	2011-11-11 16:58:00	2.89
464116	576078	23236	STORAGE TIN VINTAGE DOILY	2	2011-11-13 16:22:00	2.89
464276	576083	23236	STORAGE TIN VINTAGE DOILY	96	2011-11-14 08:22:00	2.89
464541	576112	23236	STORAGE TIN VINTAGE DOILY	6	2011-11-14 10:34:00	2.89
465677	576249	23236	STORAGE TIN VINTAGE DOILY	1	2011-11-14 13:47:00	2.89
471228	576626	23236	STORAGE TIN VINTAGE DOILY	2	2011-11-15 17:26:00	2.89
471867	576653	23236	STORAGE TIN VINTAGE DOILY	48	2011-11-16 10:45:00	2.49
			STORAGE			

475753 576911 23236 VINTAGE Description Quantity UnitPrice CustomerID
 InvoiceNo StockCode
 From the results above, the product code with the highest sales is 23843, purchased by
 InvoiceDate
 CustomerID 16446. This outcome was observed previously, and the data indicates that
 this customer purchased 80995 units for only this product. However, it was speculated
 that this transaction might have been canceled.

2011-12-09 09:00:00 581479 22081 PAPER BUNTING 1 2.95 1
 2011-12-09 09:00:00 581479 22081 WHITE LACE 1 2.95 1
 2011-12-09 09:00:00 581480 23463 VINTAGE ZINC 2 9.95 1
 2011-12-09 09:00:00 581480 23463 STORAGE WATERING CAN 2 9.95 1
 online_retail_data_table[online_retail_data_table.StockCode==23843]
 2011-12-09 09:07:00 581480 23464 WATERING CAN 4 4.95 1
 InvoiceNo StockCode Description Quantity UnitPrice CustomerID
 InvoiceDate
 2011-12-09 09:07:00 581480 23464 WATERING CAN 4 4.95 1
 2011-12-09 09:15:00 581483 23843 PAPER CRAFT, LITTLE BIRDIE 80995 2.08 16446

 2011-12-09 09:07:00 581480 22633 HAND WARMER 12 2.10 1
 2011-12-09 09:07:00 581480 22633 STORAGE UNION JACK 12 2.10 1
 2011-12-09 09:07:00 581480 22633 HOT WATER 12 2.10 1

For subsequent processing, the index is reset temporarily.

2011-12-09 09:07:00 581480 22633 HOT WATER 12 2.10 1
 online_retail_data_table.reset_index(inplace=True)
 2011-12-09 09:07:00 581480 22633 HOT WATER 12 2.10 1
 Reference Pandas Profiling, a feature of Pandas Profiling which automatically generates
 a simple data analysis report in HTML format for a given DataFrame. However, since this
 dataset is transaction data, you may get more meaningful results by filtering or
 processing the data to some extent before using Pandas Profiling. (Note that if the index
 is not reset, an error may occur)
 2011-12-09 09:07:00 581480 84347 ANGELS T-LIGHT 18 2.55 1
 2011-12-09 09:07:00 581480 84347 ANGELS T-LIGHT 18 2.55 1

```
# Install Pandas-Profiling
!pip install https://github.com/ydataai/pandas-profiling/archive/master.zip

# Downgrading the Dependency Packages of Pandas-Profiling
!pip install markupsafe==2.0.1
```

Collecting <https://github.com/ydataai/pandas-profiling/archive/master.zip>
 Downloading <https://github.com/ydataai/pandas-profiling/archive/master.zip>
 1.7 MB 24.1 MB/s 0:00:01
 Preparing metadata (setup.py) ... done
 Requirement already satisfied: scipy<1.14, >=1.4.1 in /usr/local/lib/python3.10
 Requirement already satisfied: pandas<1.4.0, >=3.1 in /usr/local/lib/python3.10
 Requirement already satisfied: matplotlib<3.10, >=3.5 in /usr/local/lib/python3.10
 Requirement already satisfied: pydantic<2 in /usr/local/lib/python3.10/dist-p
 Requirement already satisfied: PyYAML<6.1, >=5.0.0 in /usr/local/lib/python3.10
 Requirement already satisfied: Jinja2<3.2, >=2.11.1 in /usr/local/lib/python3.10
 Collecting visions<0.7.7, >=0.7.5 (from visions[image_path]<0.7.7, >=0.7.5-
 Downloading visions-0.7.6-py3-none-any.whl.metadata (11 kB)

```

Requirement already satisfied: numpy>=1.20.0 in /usr/local/lib/python3.10
2011-12-09 09:07:00 581481 23236 22495 data-profilin==0.0.dev0 2011-12-28 0.79 5.79 1
Collecting data-profilin==0.0.dev0
  Downloading data-profilin-0.0.0-py3-none-any.whl (15 kB)
Requirement already satisfied: setuptools in /usr/local/lib/python3.10/dist-packages (58.1.0)
Requirement already satisfied: typing-extensions in /usr/local/lib/python3.10/dist-packages (4.1.1)
Requirement already satisfied: tqdm>=4.48.2 in /usr/local/lib/python3.10/dist-packages (4.64.1)
Requirement already satisfied: seaborn>=0.11.1 in /usr/local/lib/python3.10/dist-packages (0.12.2)
Requirement already satisfied: matplotlib in /usr/local/lib/python3.10/dist-packages (3.5.3)
Requirement already satisfied: pandas in /usr/local/lib/python3.10/dist-packages (1.4.2)
Requirement already satisfied: scikit-learn in /usr/local/lib/python3.10/dist-packages (1.0.2)
Requirement already satisfied: statsmodels in /usr/local/lib/python3.10/dist-packages (0.13.2)
Requirement already satisfied: typeguard in /usr/local/lib/python3.10/dist-packages (2.13.3)
Requirement already satisfied: imagehash in /usr/local/lib/python3.10/dist-packages (4.3.1)
Requirement already satisfied: wordcloud in /usr/local/lib/python3.10/dist-packages (1.9.3)
Requirement already satisfied: dacite in /usr/local/lib/python3.10/dist-packages (1.8.1)
Requirement already satisfied: numba in /usr/local/lib/python3.10/dist-packages (0.56.0)
Requirement already satisfied: PyWavelets in /usr/local/lib/python3.10/dist-packages (1.8.0)
Requirement already satisfied: pillow in /usr/local/lib/python3.10/dist-packages (9.0.1)
Requirement already satisfied: MarkupSafe in /usr/local/lib/python3.10/dist-packages (2.1.1)
Requirement already satisfied: cycler in /usr/local/lib/python3.10/dist-packages (0.11.0)
Requirement already satisfied: fonttools in /usr/local/lib/python3.10/dist-packages (4.22.0)
Requirement already satisfied: kiwisolver in /usr/local/lib/python3.10/dist-packages (1.0.1)
Requirement already satisfied: packaging in /usr/local/lib/python3.10/dist-packages (20.9)
Requirement already satisfied: pyparsing in /usr/local/lib/python3.10/dist-packages (3.0.7)
Requirement already satisfied: python-dateutil in /usr/local/lib/python3.10/dist-packages (2.8.2)
Requirement already satisfied: liveliteral in /usr/local/lib/python3.10/dist-packages (0.4.4)
Requirement already satisfied: pytz in /usr/local/lib/python3.10/dist-packages (2020.1)
Requirement already satisfied: tzdata in /usr/local/lib/python3.10/dist-packages (2022.7)
Requirement already satisfied: joblib in /usr/local/lib/python3.10/dist-packages (1.1.0)
Requirement already satisfied: annotated-types in /usr/local/lib/python3.10/dist-packages (0.5.0)
Requirement already satisfied: pydantic in /usr/local/lib/python3.10/dist-packages (1.10.1)
Requirement already satisfied: matplotlib-inline in /usr/local/lib/python3.10/dist-packages (0.1.6)
Requirement already satisfied: charset-normalizer in /usr/local/lib/python3.10/dist-packages (2.0.12)
Requirement already satisfied: idna in /usr/local/lib/python3.10/dist-packages (3.4)
Requirement already satisfied: urllib3 in /usr/local/lib/python3.10/dist-packages (1.26.15)
Requirement already satisfied: certifi in /usr/local/lib/python3.10/dist-packages (2022.9.24)
Requirement already satisfied: patsy in /usr/local/lib/python3.10/dist-packages (0.5.3)
Requirement already satisfied: attrs in /usr/local/lib/python3.10/dist-packages (21.4.0)
Requirement already satisfied: networkx in /usr/local/lib/python3.10/dist-packages (2.6.3)
Requirement already satisfied: six in /usr/local/lib/python3.10/dist-packages (1.16.0)

```

```
import pandas_profiling as pdp
profile = pdp.ProfileReport(online_retail_data_table)
profile.to_file(output_file="/content/online retail data table.html")
```

2011-12-09 09:08:00	581482	23300	GARDENERS KNEELING PAD CUP OF TEA	48	1.45	1
------------------------	--------	-------	---	----	------	---

```

<ipython-input-61-4068b20210ae>:1: DeprecationWarning: `import pandas_profiling`
import pandas_profiling as pp
2011-12-09 09:08:00
Summarize dataset: 100%
44/44 [00:23<00:00, 2.68it/s, Completed]
/usr/local/lib/python3.10/dist-packages/pandas_profiling/model/pandas_summary_parser.py:581: FutureWarning: The following columns will be dropped:
series.fillna(np.nan)
LITTLE BIRDIE
80995
2.08
Generate report structure: 100%
2011-12-09 09:38:00
Render HTML: 100%
581485 22768
1/1 [00:08<00:00, 8.35s/it]
1/1 [00:01<00:00, 1.43s/it]
Export report to file: 100%
2011-12-09 09:38:00
581485 22768
1/1 [00:00<00:00, 30.49it/s]
12 9.95 1
FRAME CORNICE

```

profile

2011-12-09 09:38:00	581486	22910	PAPER CHAIN KIT VINTAGE CHRISTMAS	12	2.95	1
2011-12-09 09:38:00	581486	22086	PAPER CHAIN KIT 50'S CHRISTMAS	12	2.95	1
2011-12-09 09:38:00	581486	23344	JUMBO BAG 50'S CHRISTMAS	20	2.08	1
2011-12-09 09:38:00	581486	23351	ROLL WRAP 50'S CHRISTMAS	12	1.25	1
2011-12-09 09:38:00	581486	23352	ROLL WRAP 50'S RED CHRISTMAS	12	1.25	1
2011-12-09 09:38:00	581486	22727	ALARM CLOCK BAKELIKE RED	8	3.75	1
2011-12-09 09:38:00	581486	22730	ALARM CLOCK BAKELIKE IVORY	4	3.75	1
2011-12-09 09:38:00	581486	23203	JUMBO BAG VINTAGE DOILY	20	2.08	1
2011-12-09 09:38:00	581486	85099B	JUMBO BAG RED RETROSPOT	10	2.08	1
2011-12-09 09:38:00	581486	23201	JUMBO BAG ALPHABET	20	2.08	1
2011-12-09 09:38:00	581486	23207	LUNCH BAG ALPHABET DESIGN	10	1.65	1
2011-12-09 09:38:00	581486	22491	PACK OF 12 COLOURED PENCILS	12	0.85	1
2011-12-09 09:38:00	581486	22561	WOODEN SCHOOL	12	1.65	1

In the previous section, we performed basic data checks, basic aggregation, and fundamental analysis on CRM (POS) data. Of course, this alone does not allow us to take any actions toward customers, so from this section, we will first explain **STP**, which is often used in marketing strategies, and then learn about analytical approaches like **Decile Analysis** and **RFM Analysis**, with the goal of being able to implement them on our own.

However, this section focuses on the basic framework for analysis, so what should really be considered, as mentioned at the beginning, is what you want to achieve using the data, and it is important to clarify your objectives. In this section, the goal is to "identify customers, extract them, and use each approach as material to consider" for implementation.

If the objective is still unclear, try focusing on the three points mentioned at the beginning (Hypothesis-driven objectives, actionable insights and method comparisons). The skills required to carry out these actions are more about the framework of logical thinking, rather than simply using Python to implement or analyze data.

✓ 1-11.5.1 What is STP?

Now, let's return to the topic of marketing strategy. **STP**, which is almost always introduced in marketing books, will be explained here. STP stands for **Segment, Targeting, and Positioning**.

Why is it necessary to perform STP? As mentioned at the beginning, the customer market is not uniform, and if the same approach is used for everyone, the messaging becomes diluted for each individual. Therefore, it is crucial to properly identify customers, define the valuable ones, find potential customers, and investigate the reasons for customer churn to develop countermeasures. Implementing appropriate approaches for each segment is essential.

This leads to the discussion of segmentation and targeting. Ideally, segmentation would involve using various variables such as attribute data, geographic variables, and psychological variables, in addition to POS data. However, since we only have POS data in this case, we will explain methods for segmenting customers based on the dimensions that can be derived from POS data, such as decile analysis and RFM analysis.

The third aspect, positioning, focuses on how to make a company's products or services recognizable to the customer groups defined in the targeting phase, and how to make them desire those products or services. There are methods for analyzing the key factors

that influence customers' decisions to choose a company's products. These methods involve research or customer survey analysis. While it is difficult to perform such analysis with just POS data, those interested can look up keywords like multivariate analysis, factor analysis, and structural equations.

✓ 11-1.5.2 What is Decile Analysis?

In the previous section, we discussed segmenting customers and identifying valuable customers. However, when we talk about "valuable customers," what defines "valuable" can vary depending on the industry, company, and individual perspectives.

When we say "valuable customers," one common factor is the "amount spent." It can be said that "customers who spend a lot of money are valuable customers." But even though a customer may spend a lot, it is often unclear how much they need to spend to be classified as valuable, or at what threshold to draw the line. In the **decile analysis** introduced here, all customers are ranked in descending order of purchase amount, divided into ten equal parts, and the sales composition ratio is analyzed. For example, the top two groups might be defined as valuable customers.

The reason for conducting this analysis is that it provides a way to identify valuable customers. By identifying these customers and differentiating services from those who are not valuable, companies can encourage loyal customers to become even more devoted. It would not be beneficial to treat a customer who spends only 10 cents a year the same as a customer who spends over 100,000 dollars, as this could reduce the loyalty of high-spending customers and may even lead to a decrease in continued service usage.

Of course, in practice, customer identification and the determination of valuable customers cannot always be based solely on purchase amount, and there may be regulations that prohibit such practices in certain service areas. By adding other dimensions, such as the sales amount axis and those introduced in later sections like RFM analysis, companies can deliver messages tailored to each segment and increase the potential for offering better services.

Here, the basic procedure for segmenting is learned through decile analysis. First, we will use the data solved in the exercise above, which was the sales ranking by customer.

```
cust_total_p_rank_df.head()
```

TotalPrice	
CustomerID	
14646	280206.02
18102	259657.30
17450	194550.79
16446	168472.50
14911	143825.06

dtype: float64

To divide into 10 equal parts, we will use Pandas' `qcut`. This is a review of Pandas, so if you are not familiar with it, please review pandas.

```
q_ten_cust_price_df = pd.qcut(cust_total_p_rank_df, 10)
q_ten_cust_price_df
```

TotalPrice	
CustomerID	
14646	(3646.164, 280206.02]
18102	(3646.164, 280206.02]
17450	(3646.164, 280206.02]
16446	(3646.164, 280206.02]
14911	(3646.164, 280206.02]
...	...
17956	(-0.001, 156.566]
16454	(-0.001, 156.566]
14792	(-0.001, 156.566]
16738	(-0.001, 156.566]
13256	(-0.001, 156.566]

4339 rows × 1 columns

dtype: category

Here, let's count the number of people in each of the 10 groups and verify that they are divided equally. Use the `value_counts()` method of Pandas and sort by index in ascending order.

```
q_ten_cust_price_df.value_counts().sort_index()
```

	count
TotalPrice	
(-0.001, 156.566]	434
(156.566, 250.106]	434
(250.106, 351.658]	434
(351.658, 489.724]	434
(489.724, 674.45]	434
(674.45, 941.942]	433
(941.942, 1353.74]	435
(1353.74, 2057.914]	433
(2057.914, 3646.164]	434
(3646.164, 280206.02]	434

dtype: int64

The interpretation of the table above is as follows: There are 434 companies in the group with total sales up to 156 pounds (it might have been treated as euros during this period, but for now, we'll assume it's in pounds). The next group consists of 434 companies with sales ranging from 156 pounds to 250 pounds, and so on, with the groups divided equally.

<Practice Question 11-1-5>

1. Check if the Pareto principle (the top 20% of customers account for 80% of sales) applies.

▼ Answer

```
q_ten_cust_price_df = pd.qcut(cust_total_p_rank_df, 10, labels=['01','02','03'],
```

```
merge_cust_df = pd.concat([cust_total_p_rank_df,q_ten_cust_price_df]
                           , axis=1
                           , join='inner')
merge_cust_df.columns = ['TotalPrice','Group']
```

```
pd.DataFrame(merge_cust_df.groupby(["Group"])['TotalPrice'].sum() / merge_cust_
```

```
<ipython-input-68-9d9c01bda118>:1: FutureWarning: The default of observed=False
pd.DataFrame(merge_cust_df.groupby(["Group"])['TotalPrice'].sum() / merge_cust
```

	TotalPrice
Group	
01	0.515566
02	1.493475
03	2.976045
04	4.994334
05	7.832286
06	11.690177
07	17.223617
08	25.382863
09	38.624934
10	100.000000

From the above results, it can be seen that the total sales ratio of segment groups 1 to 8 is 25%. Furthermore, the top 20% of segment groups 9 and 10 account for about 75%, which shows that the Pareto principle holds to some extent.

✓ Side Note 7: Implementing actions after segmentation (Experimental Design)

After conducting this analysis, it is necessary to consider how to approach and implement actions for each segment (1 to 10). Since each group represents a different customer profile, it is common to apply different strategies for each. The same applies to other analyses such as RFM analysis.

To illustrate this, let us consider a scenario where the segmentation results are provided as a sales list.

For high-value customers, it is common to engage in frequent communication and offer valuable information. After segmenting, strategies for each segment are determined. However, when it comes to executing those strategies, it may sometimes be necessary for the sales representatives to review and make decisions individually for each case.

In practical terms, some financial institutions and marketing companies carry out practices similar to what is described in RFM analysis. After segmenting customers, they prioritize approaches and change the services or information offered to each group, followed by verification of the actions that resulted. Since the target customers often number in the hundreds of thousands or millions, system processing is usually employed.

This is known as experimental design. In practice, a test group within a given segment is tested with a certain strategy, while a control group is not, in order to assess the effectiveness of the strategy. This resembles A/B testing, where the uplift between the test and control groups is calculated to measure the effect of the strategy.

Specifically, for Segment 1, where few purchases have been made, a 50% discount campaign might be introduced. For Segment 10, where purchases are already high, a different service (such as a travel voucher, a gift, or information provision) might be offered. However, since the effectiveness of these changes is uncertain, "experiments" must be conducted. The results of these experiments must be verified before a PDCA (Plan-Do-Check-Act) cycle can be implemented.

However, there are instances where control cannot be established, or where legal or regulatory restrictions make it difficult to change services for customers. Approaches and methods for conducting rigorous testing when control cannot be maintained, as well as causal inference and exploratory techniques, will be introduced in more detail in the advanced-level content.

Side Note 8: Decile Analysis for Time-series

Here, the decile analysis was performed for a single period only. However, it is not always the case that the top-tier, high-value customers in segment 10 remain in segment 10 indefinitely. Additionally, there might be business intentions to move customers in segment 3 up to segment 5 through improved services or targeted actions.

To analyze this, we perform **decile time-series analysis** over two periods. The purpose of this analysis is to track the changes in sales of customers who were top-tier last year but saw a drop in sales this year, and vice versa, by observing the time-series trends of customer sales. This helps to identify the causes and take necessary actions.

Earlier, we did decile analysis for the entire period, but now we will break the data into two periods: the first half (from December 1, 2010, to July 31, 2011) and the second half (from August 1, 2011, to December 9, 2011). Given the specific data we are using, these periods are defined accordingly. In practice, it is common to define appropriate periods for analysis, such as comparing with the previous year.

Now, what trends can we find from the analysis? It is important to think about this before performing the analysis. While we do not know the business context in detail, considering possible outcomes and forming hypotheses in advance will help develop analytical thinking skills. If the business is successful, the customers in segment 10 would remain in that segment, indicating that the high-value customers are consistently engaged. We will perform the analysis with this hypothesis in mind.

<Practice Question 11-1-6>

1. Let's conduct the decile analysis for time-series based on the above settings. Before performing the analysis, consider what can be inferred from it and first hypothesize. Then, visualize how many people moved from one segment to another. Was your hypothesis correct? What other findings might there be?

✓ Answer

The most recent purchase date and the most recent maximum purchase date are as follows.

```
online_retail_data_table.InvoiceDate.min()
```

```
Timestamp('2010-12-01 08:26:00')
```

```
online_retail_data_table.InvoiceDate.max()
```

```
Timestamp('2011-12-09 12:50:00')
```

Let's divide the data into the first half and the second half. The period before July 31, 2011, will be considered the first half, and the period after that will be the second half.

```
thred_date = datetime.datetime(2011,7,31)
```

```
pre_online_retail_data_table = online_retail_data_table[online_retail_data_table.InvoiceDate < thred_date]
pos_online_retail_data_table = online_retail_data_table[online_retail_data_table.InvoiceDate >= thred_date]
```

Try the decile transition analysis on this data. To do the same thing twice, let's create a function. Also, each divided group is labeled and ranked from 1 to 10.

```
def calc_decil_func(target_cust_df):
    cust_total_p_rank_df = target_cust_df.groupby(['CustomerID'])['TotalPrice'].sort_values().rank(pct=True)
    q_ten_cust_price_df = pd.qcut(cust_total_p_rank_df, 10, labels=['01', '02', '03', '04', '05', '06', '07', '08', '09', '10'])
    return q_ten_cust_price_df
```

```
pre_q_ten_cust_price_df = calc_decil_func(pre_online_retail_data_table)
pos_q_ten_cust_price_df = calc_decil_func(pos_online_retail_data_table)
```

For the analysis setting here, we will target those who have an ID in both periods. This is because we cannot make a comparison if the person is only present in one of the periods.

```
merge_cust_prepos_df = pd.concat([pre_q_ten_cust_price_df, pos_q_ten_cust_price_df], axis=1, join='inner')
merge_cust_prepos_df.columns = ['pre', 'pos']
merge_cust_prepos_df
```

	pre	pos
CustomerID		
14646	10	10
18102	10	10
12415	10	10
17450	10	10
14156	10	10
...	...	...
17816	01	01
15885	01	04
15863	01	01
14285	01	09
16446	01	10

2000 rows × 2 columns

Based on this data, by performing the following aggregation, you can understand how individuals from the first half (upper period) moved from one decile to another in the second half. By looking at this, you can see that most people remain in almost the same segment.

```
merge_cust_prepos_df.groupby(['pre', 'pos'])['pos'].count().unstack()
```

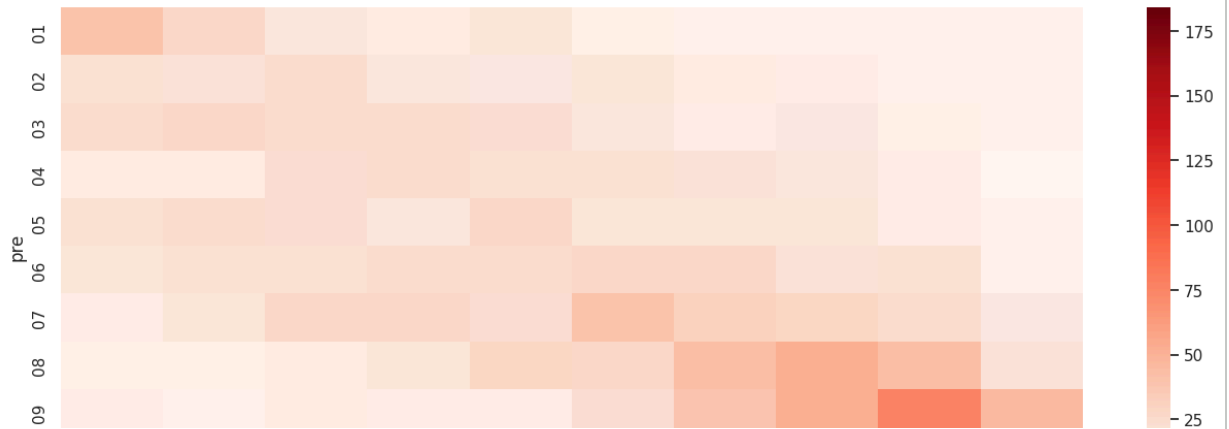
```
<ipython-input-76-2c4be9673e0d>:1: FutureWarning: The default of observed=False
merge_cust_prepos_df.groupby(['pre', 'pos'])['pos'].count().unstack()
```

pos	01	02	03	04	05	06	07	08	09	10
pre										
01	41	28	13	9	16	6	3	3	3	2
02	22	18	25	13	12	16	9	7	4	3
03	25	26	24	25	23	14	8	12	6	3
04	9	11	23	25	22	21	20	15	8	0
05	22	24	23	14	26	17	16	16	8	4
06	16	21	21	25	24	27	28	20	21	3
07	8	16	28	28	23	40	32	29	24	12
08	6	5	11	16	29	26	43	53	44	18
09	7	2	11	8	7	23	39	53	77	45
10	3	0	2	2	4	5	14	23	51	184

To make it visually easier to understand, the following will display the data as a heatmap.

```
plt.figure(figsize=(16,6))
sns.heatmap(merge_cust_prepos_df.groupby(['pre', 'pos'])['pos'].count().unstack(
```

```
<ipython-input-77-c169eefdef6d>:2: FutureWarning: The default of observed=False
sns.heatmap(merge_cust_prepos_df.groupby(['pre', 'pos'])['pos'].count().unstack()
<Axes: xlabel='pos', ylabel='pre'>
```



Visually, this makes it easier to understand, and we can see that the number of people is higher around Segment 10. This suggests that the business is successfully capturing valuable customers. On the other hand, there are still quite a few customers remaining in Segment 1, so it might be a good idea to consider a customer journey strategy to move these customers to higher segments.

This concludes the answer to the practice question. Here, we only tracked the movement of deciles numerically, but as mentioned earlier, it is important to implement initiatives aimed at moving customers up within a particular decile segment and then observe how they move in the next half-year, checking whether the number of people who moved up has increased, through experimentation and verification. Decile analysis and the subsequent RFM analysis cannot directly provide actionable insights just by themselves, so it's essential to be cautious.

Also, while we performed a simple analysis by dividing the periods, in many cases, analysis may require narrowing down the target group, such as including only people with IDs during a specific period across both the first and second half. In fact, this approach is more typical when examining changes in customer behavior, and how you design your analysis becomes crucial when performing such an analysis.

✓ 11-1.5.3 What is ABC Analysis?

So far, we have focused on identifying and segmenting key customers, but now we will shift our attention to products and introduce **ABC analysis**, also known as priority analysis. The basic approach is similar to decile analysis, but instead of focusing on customers, we analyze products. ABC analysis is used to evaluate products, for example, to identify best-sellers and slow-moving items.

In ABC analysis, products are categorized as follows: A accounts for up to 80% of cumulative share, B for up to 95%, and C for the rest. These thresholds can be adjusted. The concept is similar to the **Pareto principle**, and the analysis helps determine whether A and B products should be prioritized for handling.

As an additional note, ABC analysis is often conducted within product categories. For instance, analysis might be performed specifically on the beer category within alcoholic beverages or on a particular category of cosmetics, such as lipsticks. Narrowing down the product scope is a common practice.

Regarding category classification, it is typically predefined but not entirely fixed. Whether a product is categorized as A or B may depend on the analyst's judgment. Often, a product category master is defined before analysis. If you understand decile analysis or the Pareto principle, ABC analysis is straightforward. However, since we lack a product category master in this case, we will conduct ABC analysis on all products. When a product category master is available, it can simply be joined with the results afterward, so the implementation remains fundamentally unchanged. (Note: While defining product categories can be challenging, some companies offer solutions that efficiently map categories.)

First, calculate the total sales amount for each product.

```
stock_total_p_rank_df = online_retail_data_table.groupby(['StockCode'])['TotalF  
stock_total_p_rank_df.head()
```

	TotalPrice
StockCode	
23843	168469.60
22423	142592.95
85123A	100603.50
85099B	85220.78
23166	81416.73

dtype: float64

For simplicity, we will sample the data (using 100 products) and perform the ABC analysis. This implementation is merely a sample and holds no significant meaning. As mentioned earlier, in practice, it is necessary to create category master data before conducting the analysis. For instance, when the author was analyzing large-scale POS data at a marketing company, the sales representatives created master data defining broad and subcategories for each category, which was then used for analysis. Since we do not have such master data for this exercise, we will omit it.

```
sample_df = stock_total_p_rank_df.sample(n=100)
sample_df.sort_values(ascending=False, inplace=True)
```

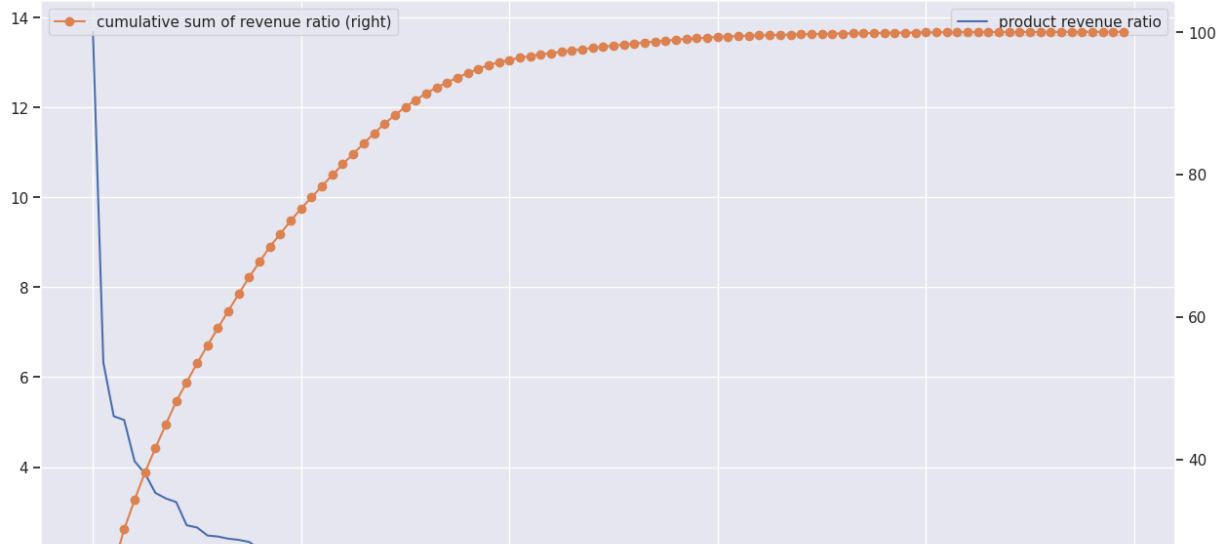
Next, let's calculate the cumulative sales and their overall proportions.

```
x = sample_df/sample_df.sum() * 100
```

```
y = sample_df.cumsum() / sample_df.sum() * 100
```

```
plt.figure(figsize=(15,9))
x.plot(label='product revenue ratio')
plt.legend()
y.plot(marker='o',secondary_y=True,label='cumulative sum of revenue ratio')
plt.legend()
```

<matplotlib.legend.Legend at 0x79901a034970>



In the graph above, the horizontal axis represents products ranked by their sales, the left vertical axis shows the proportion of each product's sales relative to the total as represented by the blue line, and the right vertical axis depicts the cumulative sales proportion of each product, shown with the orange dotted line. This forms the basis for the ABC analysis.

Focusing on the orange line, it can be observed that approximately 20 percent of the top-ranked products account for around 80 percent of the total sales, roughly aligning with the Pareto Principle.

While the implementation above was introduced to help understand the logic behind ABC analysis, it can be difficult for identifying ranges for A, B, and C products. Although it is feasible to implement, it becomes easier to use a library designed for ABC analysis. Let us install and use this library to simplify the process.

Install the `abc_analysis` library.

```
!pip install abc_analysis
```

```
Collecting abc_analysis
  Downloading abc_analysis-0.1.23-py3-none-any.whl.metadata (1.9 kB)
Requirement already satisfied: pandas>=0.22 in /usr/local/lib/python3.10/dist-pa
Requirement already satisfied: numpy>=1.14 in /usr/local/lib/python3.10/dist-pac
Requirement already satisfied: scipy>=1.1.0 in /usr/local/lib/python3.10/dist-pa
```

```
Requirement already satisfied: matplotlib>=2.2.2 in /usr/local/lib/python3.10/di
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.10/dis
Requirement already satisfied: cyclor>=0.10 in /usr/local/lib/python3.10/dist-pa
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.10/di
Requirement already satisfied: kiwisolver>=1.0.1 in /usr/local/lib/python3.10/di
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.10/dist
Requirement already satisfied: pillow>=6.2.0 in /usr/local/lib/python3.10/dist-p
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.10/dis
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.10
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.10/dist-pa
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.10/dist-
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.10/dist-packag
Downloading abc_analysis-0.1.23-py3-none-any.whl (17 kB)
Installing collected packages: abc_analysis
Successfully installed abc_analysis-0.1.23
```

Import `abc_analysis` and `abc_plot` from the `abc_analysis` library.

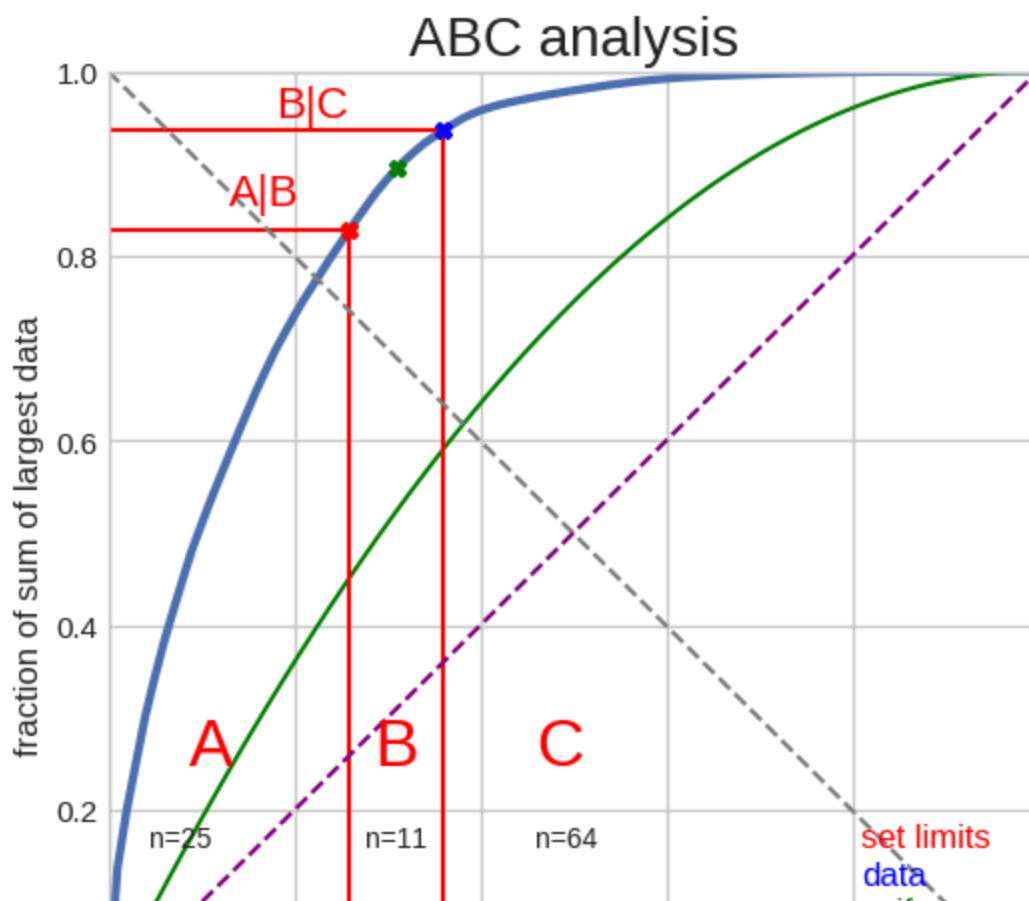
```
from abc_analysis import abc_analysis, abc_plot
```

The input for `abc_analysis` should consist of data where the product key is associated with sales. After inputting the data, the calculation will be complete.

```
abc_analysis_result = abc_analysis(sample_df)
```

By using `abc_plot`, the following ABC analysis results can be displayed. The vertical axis represents the proportion of total sales, and the horizontal axis shows the proportion of sales for each product. From this, you can observe that approximately 20% of the products account for 80% of the sales, which forms the A group.

```
abc_plot(abc_analysis_result)
```



just by looking at this diagram, it is not possible to tell which part is group A, which part is group B, and so on. Therefore, as follows, the `abc_analysis_result` holds data in the form of a dictionary, and when you want to check values like the thresholds above, you can specify the keys to retrieve them.

```
type(abc_analysis_result)
```

```
dict
```

```
abc_analysis_result.keys()
```

```
dict_keys(['ABLimit', 'BCLimit', 'ABexchanged', 'Aind', 'Bind', 'Cind',
'smallestAData', 'smallestBData', 'AlimitIndInInterpolation',
'BlimitIndInInterpolation', 'p', 'ABC', 'A', 'B', 'C', 'CleanedData'])
```

For example, the thresholds for the amounts between A and B, and between B and C, are as follows. If you want to extract only the products from A, you can extract the products

with a sales amount greater than or equal to the threshold below.

```
print('The amount threshold between A and B is {0:.1f}'.format(abc_analysis_re  
print('The amount threshold between B and C is {0:.1f}'.format(abc_analysis_re
```

```
The amount threshold between A and B is 5823.4.  
The amount threshold between B and C is 2691.9.
```

```
print('The group of products from A is as follows.')  
sample_df[sample_df>=abc_analysis_result['ABLimit']]
```

The group of products from A is as follows.

TotalPrice	
StockCode	
M	53779.93
22699	24865.60
22411	20174.23
21731	19844.91
22865	16237.63
84997B	15158.81
48184	13470.95
23108	12985.23
22752	12665.29
22413	10647.30
47566B	10445.25
23343	9748.34
22569	9655.48
22692	9468.08
22766	9365.78
22978	9184.28
21791	8531.41
22989	8323.17
84949	7193.25
23238	7170.60
22333	6749.23
22826	6210.00
21871	6151.78
23480	6126.09
71053	5915.95
47591D	5823.36

dtype: float64

This concludes the introduction to the ABC analysis. In terms of applying this ABC analysis, for example, when deciding which products to strengthen and which products to reduce in order to increase sales or cut procurement costs, the analysis can be used as one of the considerations for whether to strengthen products from group A or B, or reduce procurement for products from group C. Additionally, after performing decile analysis and the RFM analysis, which will be introduced later, it is possible to carry out product ABC analysis within each customer segment and consider providing products and services tailored to the needs of each segment.

<Practice Question 11-1-7>

1. Extract only the products from group C.
2. Extract only the products from group B.
3. Focus on the two countries (Australia and Germany), conduct the ABC analysis for each, and examine if there are any differences between the AB groups.

✓ Answer

```
print('The group of products from C is as follows.')
sample_df[sample_df<=abc_analysis_result['BCLimit']]
```


The group of products from C is as follows.

```
print('The group of products from B is as follows.')
sample_df[(sample_df<abc_analysis_result['ABLimit'])&(sample_df>abc_analysis_re
```

The group of products from B is as follows.

StockCode	TotalPrice
22843	2691.88
23489	2512.44
22845	2401.02
22971	5603.57
21999	1904.00
21156	5587.87
22244	1602.01
23531	5340.21
20001	1199.90
21172	4708.27
21902	1176.72
21390	4544.13
23199	540.07
72351B	3860.24
22502	828.02
22596	3567.70
20100	510.40
22908	3330.64
22442	762.20
22746	2733.02
21120	171.00
22586	676.60

dtype: float64

21991	675.53
-------	--------

Let us calculate the total sales based on the product code and country. Note that purchasing data from e-commerce sites, for example, is often represented in a matrix like the one below, and is usually sparse (with many NaN values).

21894	610.73
-------	--------

```
s_c_tp_df = online_retail_data_table.groupby(['StockCode', 'Country'])['TotalPri
s_c_tp_df.head(20)
```

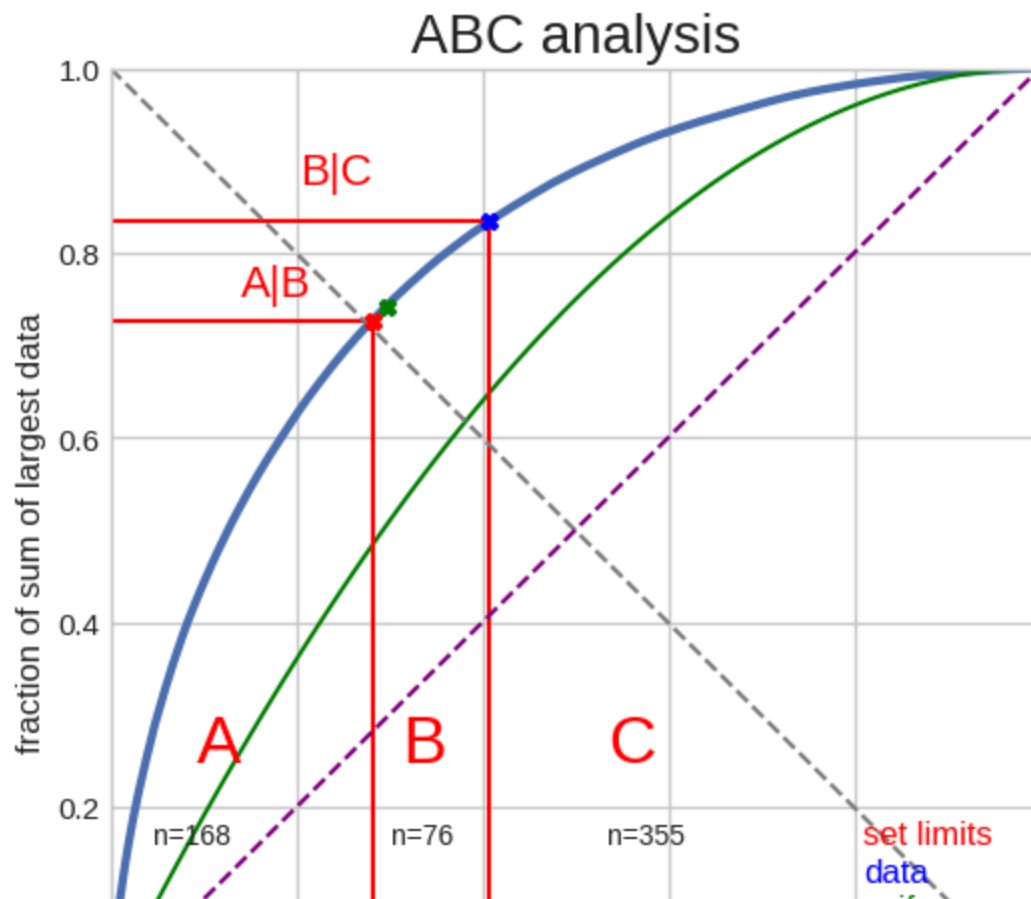
21000	501.00
84816	519.35
22116	493.39
23484	483.66
21358	410.00
16011	372.33
21635	367.50
84951B	318.44
22346	309.69
20782	221.36
85074B	205.00

90060D 17.85
t and outputs the result of the ABC analysis.
21661 17.85
16248B 16.25

```
def country_abc_analysis(country_name, target_df):
    sample_df = target_df[country_name].dropna()
    abc_analysis_result = abc_analysis(sample_df)
    return abc_analysis_result
```

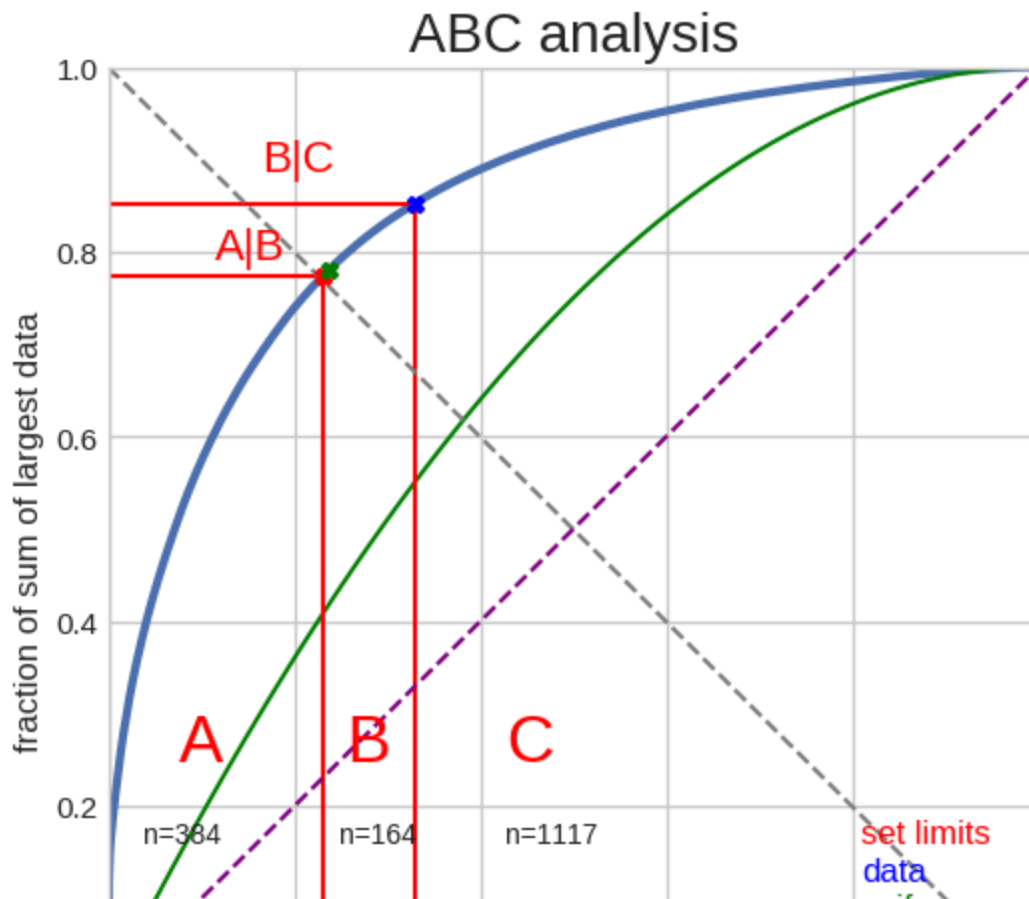
The data for both Australia and Germany is presented. Below is the data for Australia.

```
abc_analysis_result_aud = country_abc_analysis('Australia', s_c_tp_df)
abc_plot(abc_analysis_result_aud)
```



Next is the data for Germany.

```
abc_analysis_result_aud = country_abc_analysis('Germany', s_c_tp_df)
abc_plot(abc_analysis_result_aud)
```



From the above comparison, it can be observed that in Australia, 40% of the products account for 80% of the sales, while in Germany, slightly over 30% of the products make up a similar proportion. This suggests that products in Germany may be selling in a more concentrated manner.

Additionally, if there are any hypotheses or other perspectives, different from those in the practice questions, feel free to explore them.

✓ 11-1.5.4 What is RFM Analysis?

Now, let's shift the focus back to customers. In decile analysis, customers were classified based solely on their purchasing amounts. However, the question of whether customers who spend more are always better or more valuable is a separate issue.

For instance, even if two customers both spend \$10,000, the approach will differ depending on whether they made the purchase within the last month or ten years ago.

Similarly, a customer who spends \$10,000 in a single purchase behaves differently from one who spends \$100 spread over 100 transactions.

Therefore, here we will introduce a method called **RFM analysis** to more appropriately segment valuable customers for your business, going beyond just total purchase amounts. The RFM analysis stands for **Recency (the most recent purchase date), Frequency (total number of purchases), and Money (total amount spent)**, and is an approach to classify customers based on these three dimensions. The more a customer spends, the more recent their purchases, and the higher their purchase frequency, the more they are considered a valuable customer, and this approach helps identify them.

In RFM analysis, thresholds are defined for each of the three dimensions. For Money, for example, customers are segmented into three groups: those with low purchase amounts, those with moderate purchases, and those with high purchases. Specifically, customers who have a total amount from 0 to 100 can be classified as Rank 1, those with a total between 101 and 1000 as Rank 2, and those above 1000 as Rank 3.

There is no absolute rule for setting these thresholds, and RFM analysis can vary depending on the industry or company, with variations such as using only RF or FM.

Here, let's assume that the data was extracted on the reference date, December 10, 2011, and proceed with the analysis.

Since the Frequency (F) and Money (M) have already been analyzed and aggregated, we will now focus on calculating the Recency (R). Specifically, we will calculate how much time has passed since the last purchase, relative to the reference date of December 10, 2011.

```
base_date = datetime.datetime(2011,12,10)
```

To extract the most recent purchase date for each customer, we will first reset the index.

```
online_retail_data_table.reset_index(inplace=True)
```

The most recent purchase date for each customer is being calculated as shown below.

```
cust_rec_rank_df = online_retail_data_table.groupby(['CustomerID'])['InvoiceDate'].max().reset_index()
```

InvoiceDate	
CustomerID	
12680	2011-12-09 12:50:00
13113	2011-12-09 12:49:00
15804	2011-12-09 12:31:00
13777	2011-12-09 12:25:00
17581	2011-12-09 12:21:00
...	...
16583	2010-12-01 12:03:00
17908	2010-12-01 11:45:00
12791	2010-12-01 11:27:00
13747	2010-12-01 10:37:00
18074	2010-12-01 09:53:00

4339 rows × 1 columns

dtype: datetime64[ns]

Next, we will calculate how many days have passed since the reference date. To include that day, we add 1 day, and to calculate in terms of days, we convert the result to a numerical value using `dt.days`.

```
cust_rec_rank_df = (base_date - cust_rec_rank_df + datetime.timedelta(days=1)).
cust_rec_rank_df
```

InvoiceDate	
CustomerID	
12680	1
13113	1
15804	1
13777	1
17581	1
...	...
16583	374
17908	374
12791	374
13747	374
18074	374

4339 rows × 1 columns

dtype: int64

From the above, we can see that some customers made purchases as recently as the previous day, while others have not made any purchases for over a year.

Next, to perform the RFM analysis, we will first consolidate the data by CustomerID. The results from the previous practice questions will also be used, with CustomerID as the key.

```
merge_cust_rmf_df = pd.concat([cust_rec_rank_df, cust_invoice_rank_df, cust_tot
                               , axis=1
                               , join='inner')
merge_cust_rmf_df.columns = ['R', 'F', 'M']
merge_cust_rmf_df
```

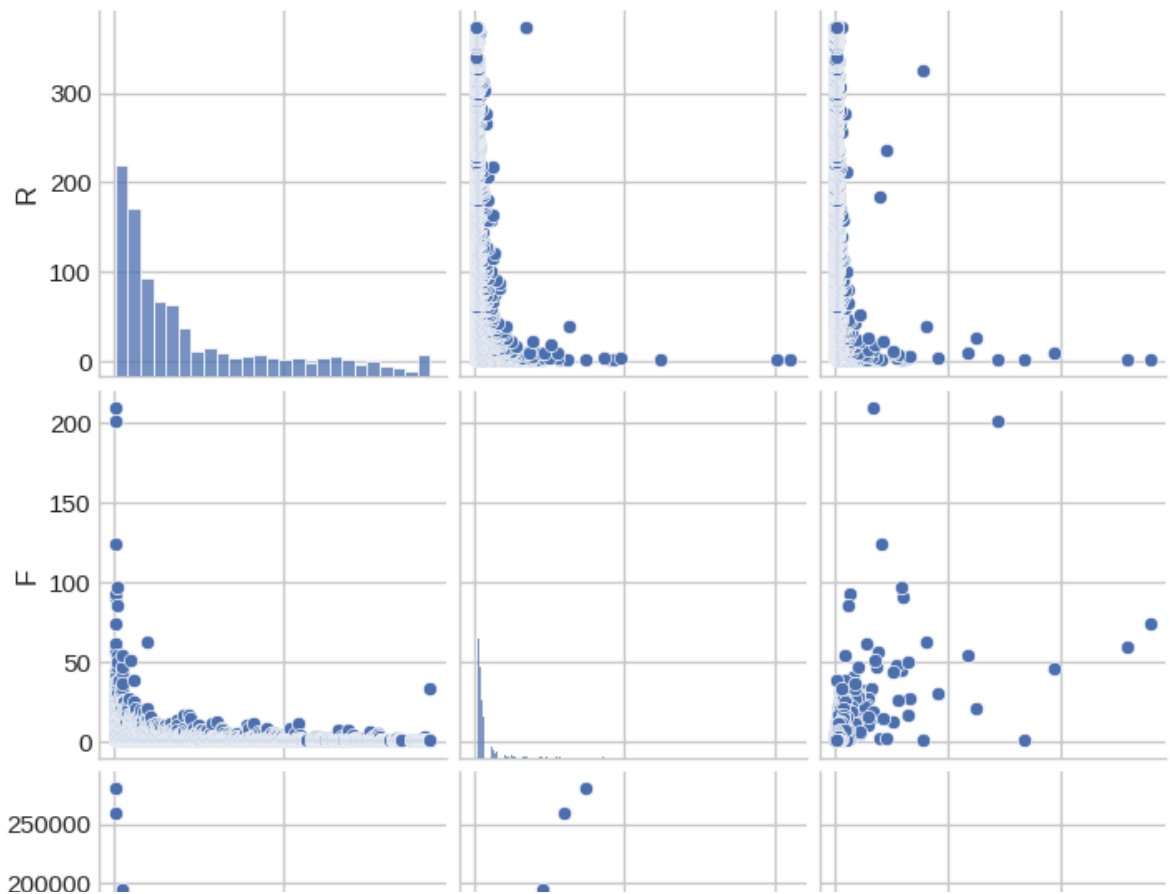
	R	F	M
CustomerID			
12680	1	4	862.81
13113	1	24	12245.96
15804	1	13	4206.39
13777	1	33	25977.16
17581	1	25	11045.04
...	...	...	...
16583	374	1	233.45
17908	374	1	243.28
12791	374	1	192.60
13747	374	1	79.60
18074	374	1	489.60

4339 rows × 3 columns

Let's use Seaborn's pairplot to check the scatter plots and histograms for each of the variables. (You can review about data visualization using materials from week 4)

```
sns.pairplot(merge_cust_rmf_df)
```

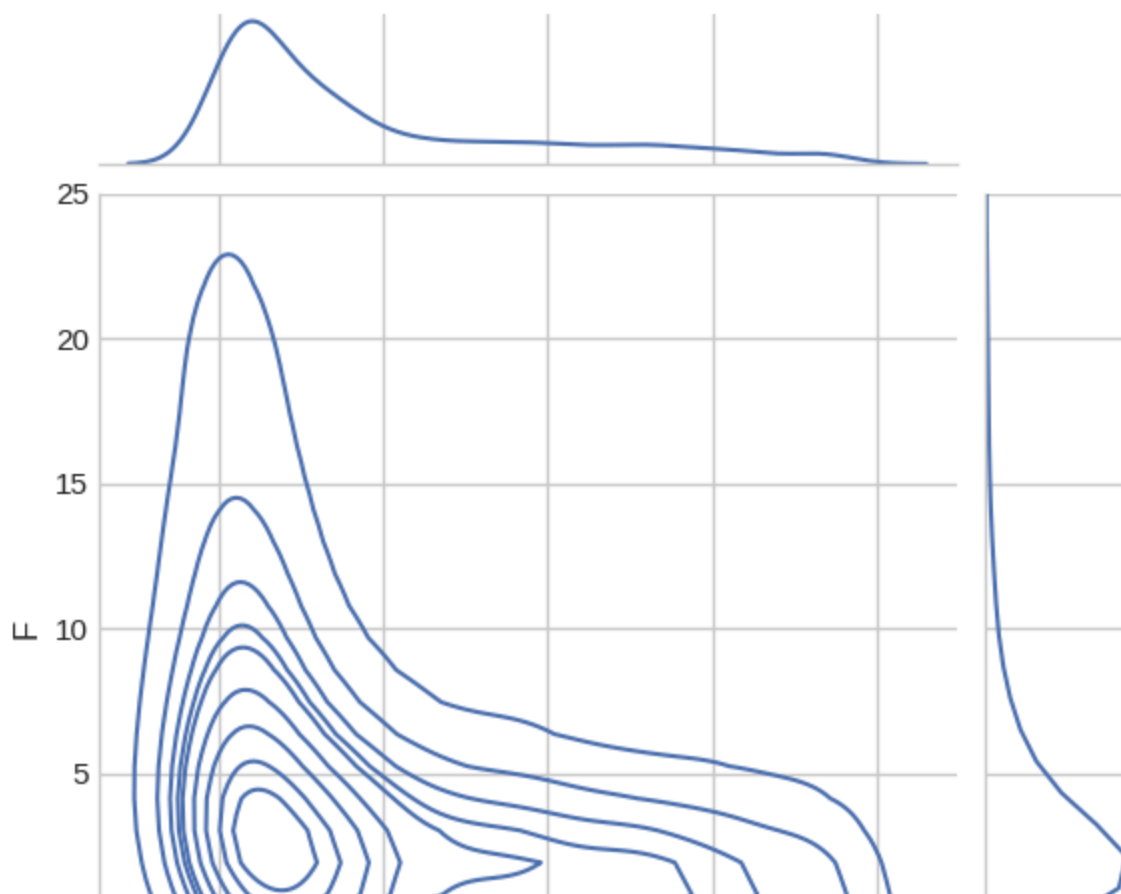
```
<seaborn.axisgrid.PairGrid at 0x79901b5094b0>
```



Looking at the above graphs, we can see that the Recency (R) distribution is long-tailed. Similarly, Frequency (F) and Money (M) also show a long tail, but compared to R, the tail is much wider, indicating a clear division between customers who make frequent purchases and those who do not. This phenomenon is commonly observed in e-commerce or purchasing-related data across many platforms.

However, the relationship between R and F is not very clear from the above graph. Let's use `jointplot` to examine this in more detail. This will allow us to see the volume zones on both axes more clearly.

```
ax = sns.jointplot(x="R", y="F", kind="kde", data=merge_cust_rmf_df)
plt.ylim(-5,25)
plt.show()
```



The relationship between R and M also follows a similar pattern as shown above. (There are significant negative values for R and F, but please aware that this is due to the results "estimated" by the kernel density function.)

Here, we will decide to split the data based on the thresholds for each variable. However, since it is quite difficult to split F evenly, so we will proceed with dividing it using the following method.

```
q_f_merge_cust_rmf_df = pd.cut(merge_cust_rmf_df.F, [0, 1, 3, 10, 100,250])
```

```
q_f_merge_cust_rmf_df.value_counts().sort_index()
```

	count
F	
(0, 1]	1494
(1, 3]	1343
(3, 10]	1165
(10, 100]	334
(100, 250]	3

dtype: int64

For the purchase amount and frequency, we will also divide them into 5 equal segments using the following method.

```
q_r_merge_cust_rmf_df = pd.qcut(merge_cust_rmf_df.R, 5)
q_m_merge_cust_rmf_df = pd.qcut(merge_cust_rmf_df.M, 5)
```

Let's check how each of the segments has been divided.

```
q_r_merge_cust_rmf_df.value_counts().sort_index()
```

	count
R	
(0.999, 14.2]	868
(14.2, 33.0]	881
(33.0, 72.0]	863
(72.0, 180.0]	866
(180.0, 374.0]	861

dtype: int64

```
q_m_merge_cust_rmf_df.value_counts().sort_index()
```

	count
M	
(-0.001, 250.106]	868
(250.106, 489.724]	868
(489.724, 941.942]	867
(941.942, 2057.914]	868
(2057.914, 280206.02]	868

dtype: int64

Next, we have merged all the data for the RFM analysis.

```
q_merge_cust_rmf_df = pd.concat([q_r_merge_cust_rmf_df, q_f_merge_cust_rmf_df,
                                , axis=1
                                , join='inner')
q_merge_cust_rmf_df.columns = ['gR', 'gF', 'gM']
q_merge_cust_rmf_df
```

	gR	gF	gM
CustomerID			
12680	(0.999, 14.2]	(3, 10]	(489.724, 941.942]
13113	(0.999, 14.2]	(10, 100]	(2057.914, 280206.02]
15804	(0.999, 14.2]	(10, 100]	(2057.914, 280206.02]
13777	(0.999, 14.2]	(10, 100]	(2057.914, 280206.02]
17581	(0.999, 14.2]	(10, 100]	(2057.914, 280206.02]
...	...	...	...
16583	(180.0, 374.0]	(0, 1]	(-0.001, 250.106]
17908	(180.0, 374.0]	(0, 1]	(-0.001, 250.106]
12791	(180.0, 374.0]	(0, 1]	(-0.001, 250.106]
13747	(180.0, 374.0]	(0, 1]	(-0.001, 250.106]
18074	(180.0, 374.0]	(0, 1]	(250.106, 489.724]

4339 rows × 3 columns

Let's also include the actual values for the segments.

```
all_merge_cust_rmf_df = pd.concat([merge_cust_rmf_df,q_merge_cust_rmf_df]
                                   , axis=1
                                   , join='inner')

all_merge_cust_rmf_df
```

	R	F	M	gR	gF	gM
CustomerID						
12680	1	4	862.81	(0.999, 14.2]	(3, 10]	(489.724, 941.942]
13113	1	24	12245.96	(0.999, 14.2]	(10, 100]	(2057.914, 280206.02]
15804	1	13	4206.39	(0.999, 14.2]	(10, 100]	(2057.914, 280206.02]
13777	1	33	25977.16	(0.999, 14.2]	(10, 100]	(2057.914, 280206.02]
17581	1	25	11045.04	(0.999, 14.2]	(10, 100]	(2057.914, 280206.02]
...	...	...	...	...	...	...
16583	374	1	233.45	(180.0, 374.0]	(0, 1]	(-0.001, 250.106]
17908	374	1	243.28	(180.0, 374.0]	(0, 1]	(-0.001, 250.106]
12791	374	1	192.60	(180.0, 374.0]	(0, 1]	(-0.001, 250.106]
13747	374	1	79.60	(180.0, 374.0]	(0, 1]	(-0.001, 250.106]
18074	374	1	489.60	(180.0, 374.0]	(0, 1]	(250.106, 489.724]

4339 rows × 6 columns

In RFM analysis, it is common to visualize the data in a three-dimensional graph. However, this can become somewhat difficult to interpret, so we will decide to aggregate the data along two axes instead. Since F and M are likely to be highly correlated (with basket count and purchase amount being related), we will initially select M and then choose R as the remaining axis. Below is a table showing the number of customers in each segment for both R and M.

```
all_merge_cust_rmf_df.groupby(["gM","gR"])["M"].count().unstack()
```

```
<ipython-input-111-c5b8e22a0e79>:1: FutureWarning: The default of observed=False
all_merge_cust_rmf_df.groupby(["gM", "gR"])["M"].count().unstack()
```

	gR	(0.999, 14.2]	(14.2, 33.0]	(33.0, 72.0]	(72.0, 180.0]	(180.0, 374.0]
gM						
(-0.001, 250.106]		59	108	160	182	359
(250.106, 489.724]		77	137	173	197	284
(489.724, 941.942]		125	177	200	232	133
(941.942, 2057.914]		211	230	189	174	64
(2057.914, 280206.02]		396	229	141	81	21

In this case, the customers become more valuable as they move towards the lower-left corner. Based on the results above, it appears that many customers have a relatively low purchase amount and have not made recent purchases.

Next, calculate the total sales amount for each segment.

```
all_merge_cust_rmf_df.groupby(["gM", "gR"])["M"].sum().unstack()
```

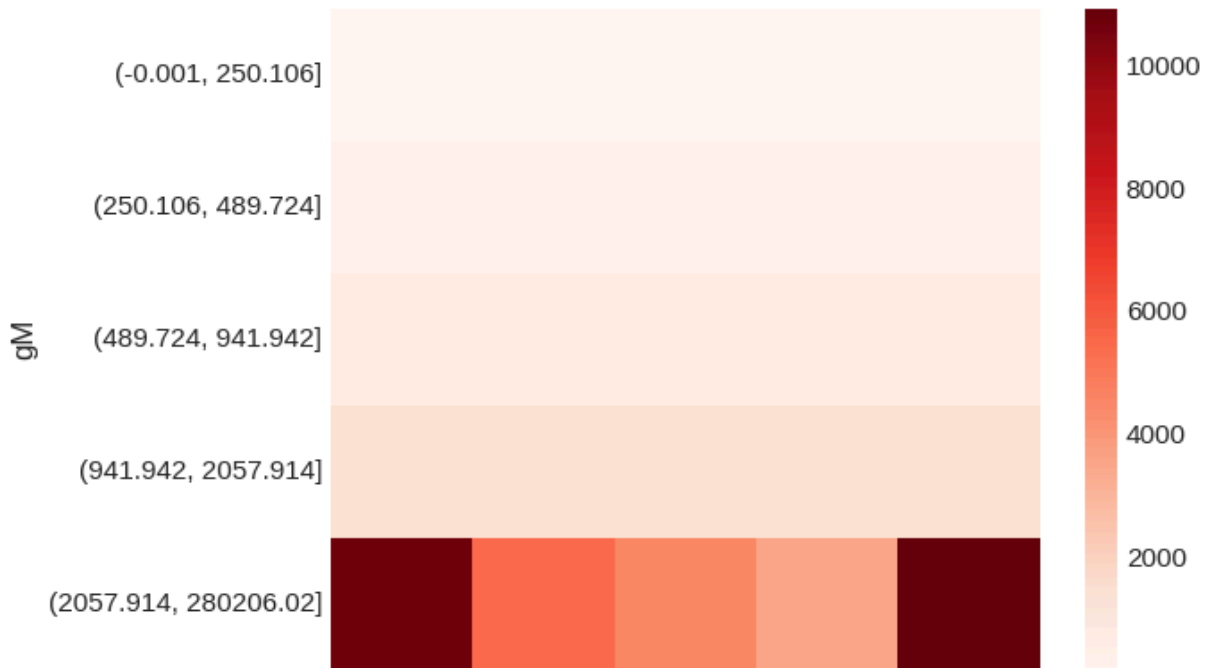
```
<ipython-input-112-d0f3e09708d6>:1: FutureWarning: The default of observed=False
all_merge_cust_rmf_df.groupby(["gM", "gR"])["M"].sum().unstack()
```

	gR	(0.999, 14.2]	(14.2, 33.0]	(33.0, 72.0]	(72.0, 180.0]	(180.0, 374.0]
gM						
(-0.001, 250.106]		10156.96	16419.180	25963.91	27299.450	53250.140
(250.106, 489.724]		28176.81	49630.030	63659.04	70889.421	99620.520
(489.724, 941.942]		88965.61	123845.151	137183.89	158086.111	88613.130
(941.942, 2057.914]		305244.63	324123.210	265522.90	239798.480	85521.870
(2057.914, 280206.02]		4231690.91	1257109.360	641092.08	290138.090	229407.021

The following is a heatmap representing the purchase amount per company. As expected, higher ranks for M correspond to larger amounts. However, what we can also observe is that there are loyal customers who made recent purchases, as well as customers who, despite not having made a purchase in over 180 days, had made significant purchases in the past.

```
sns.heatmap(all_merge_cust_rmf_df.groupby(["gM", "gR"])["M"].sum().unstack() / a
```

```
<ipython-input-113-f8b42ba23c7d>:1: FutureWarning: The default of observed=False
sns.heatmap(all_merge_cust_rmf_df.groupby(["gM", "gR"])["M"].sum().unstack() /
<Axes: xlabel='gR', ylabel='gM'>
```



From this, it is important to consider hypotheses about why customers who made significant purchases in the past have stopped buying recently. However, without more background information or data, it is difficult to determine the exact reason. For example, they might be customers who don't buy gifts frequently but make regular purchases every six months to a year, or they might have been a one-time customer due to a specific campaign. Nonetheless, it is worth considering an approach to reach out to customers who have not made recent purchases. What works best can be tested through experiments and verified by checking whether the situation improves.

This concludes the RFM analysis. As mentioned in the Decile analysis, the key is not to stop here. For each segment, it is important to change the approach and conduct experiments to test whether these changes were effective or not. The real value comes from continuously experimenting and verifying the results.

<Practice Question 11-1-8>

1. Perform the RFM analysis by changing the thresholds in various ways.
2. Select several countries and perform the RFM analysis for each country.

(The implementation can be done by changing the thresholds or filtering the original data by selecting countries, so the answer is omitted.)

✓ 11-1.5.5 What is Churn Analysis?

Until now, the analysis has focused on existing customers. The objective has been to segment customers and identify them in order to improve the service provided to these existing customers.

Here, I will introduce **Churn analysis**. As the name suggests, churn analysis is conducted to understand why customers may leave (stop making purchases) and to take measures to address this issue. While a certain level of **churn** is unavoidable, a growing number of customers leaving can significantly impact a company's revenue. Therefore, it is crucial to quickly detect this change, understand why customers are churning, and determine how to prevent it.

Just like identifying customers, churn analysis aims to extract customers with the goal of preventing churn. The specific implementation of churn analysis can actually be done using the same approach as the decile analysis. As a KPI, the **retention rate** (1 - churn

rate) is often set and monitored, so churn analysis can also be considered as analysis of retention.

Here, there are some important points to note about customer churn analysis. For services that involve ongoing contracts, such as mobile phone or insurance services, it is easier to define "churn" based on the data. On the other hand, for consumer goods (such as beer or daily necessities), it can be difficult to determine whether a customer is continuing to purchase or not. In such cases, it is necessary to define churn by setting a certain purchase period. If there is no purchase during that period, the customer is considered to have churned. In this case, we will define churn based on a set period.

So, how do we specifically define "churn" in the data? Before looking at the implementation below, think about it for a moment.

The data used in the decile analysis will be reused below.

```
merge_cust_prepos_df = pd.concat([pre_q_ten_cust_price_df, pos_q_ten_cust_price
                                , axis=1
                                , join='outer')
merge_cust_prepos_df.columns = ['pre', 'pos']
merge_cust_prepos_df
```

	pre	pos
CustomerID		
14646	10	10
18102	10	10
12415	10	10
12346	10	NaN
17450	10	10
...	...	...
13307	NaN	01
16878	NaN	01
16454	NaN	01
14792	NaN	01
13256	NaN	01

4339 rows × 2 columns

It was divided into pre (first half) and pos (second half). From the data above, we can see that there were originally the following number of people in the Pre (first half).

```
pre_count = merge_cust_prepos_df.pre.notnull().sum()
pre_count
```

3144

Churn here refers to customers who were present in the first half of the year (pre) but are no longer present in the second half of the year (pos). In the data, those who were not NaN in the pre data but became NaN in the pos data are considered churned. The number of churned customers is calculated below.

The conditions are filtered as described above, and the calculation is based on this filtering.

```
pos_churn_count = merge_cust_prepos_df[merge_cust_prepos_df.pre.notnull()].pos.
pos_churn_count
```

1144

```
pos_churn_count / pre_count * 100
```

```
36.38676844783715
```

It cannot be said that this churn rate is necessarily high, but by regularly monitoring the time series of this churn rate (or retention rate), it becomes clear whether the churn rate is rising or being suppressed. If the churn rate is increasing and becomes a problem, it is important to monitor this number continuously as a KPI and, for example, set a goal to reduce the churn rate from 36% to 25% within the year, then devise strategies and measures to achieve this. The KPI reporting tools for this can be created using Python, and in the next advanced session, we will briefly introduce how to automate such tasks.

Additionally, since the products sold on this e-commerce site are "gifts," it is understandable that the purchasing behavior differs from that of regular consumer goods (such as beverages, snacks, and daily necessities). In that sense, even if the churn rate is higher than that of consumer goods, it can be justified. In such cases, let's consider the comparison targets, design, and analyze accordingly.

<Practice Question 11-1-9>

Below, without changing the definition of churn, please investigate whether there is any difference in the churn rate of customers (CustomerID) by country.

✓ Answer

The following checks which customer made purchases in which country. Some customers have made purchases in two countries, but since the number of such customers is small, they will be excluded from this analysis.

```
cutum_country_cnt_df = online_retail_data_table.groupby(["CustomerID"])[ "Country"]  
cutum_country_cnt_df.head(10)
```

Country	
CustomerID	
12417	2
12429	2
12455	2